



Cours Complet de Bases de Données

Approche théorique et pratique

 **Module:** Bases de Données

 **Réalisé par:** Youssef EL MANSOURI

 **Groupe:** 2

 **Année:** 2025/2026

 **Encadré par:** M. Mouhsine LAKHDISSI



Table des Matières

1 Introduction aux Bases de Données

3 Modélisation Conceptuelle des Données

5 Normalisation des Bases de Données

7 SQL – Langage de Définition de Données

9 SQL – Langage de Requête de Données

11 SGBD Oracle

13 Sécurité et Administration des Bases de Données

2 Systèmes de Gestion de Bases de Données (SGBD)

4 Modèle Relationnel

6 SQL – Introduction Générale

8 SQL – Langage de Manipulation de Données

10 SGBD MySQL

12 PL/SQL (Programmation en Base de Données)

14 Révisions et Études de Cas



CHAPITRE 1

INTRODUCTION AUX BASES DE DONNÉES

Cours de Bases de Données



Information



Connaissance



Évolution

1.1 Donnée, Information et Connaissance

Donnée

Élément brut, sans interprétation directe

- Ahmed
- 25
- INF102
- 14/09/2025

→ Ces valeurs n'ont pas de sens complet

Information

Donnée traitée, organisée et contextualisée

Ahmed, âgé de 25 ans, est inscrit au module INF102 le 14/09/2025

→ Devenues compréhensibles et exploitables

Connaissance

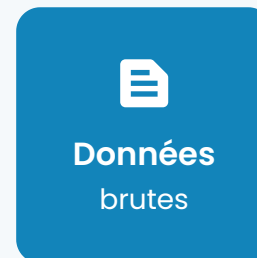
Information analysée, comparée et interprétée → utile pour la prise de décision

✓ Analysée

✓ Comparée

✓ Interprétée

Processus de transformation



↓
Traitement



↓
Analyse



1.2 Historique de la gestion des données

Gestion traditionnelle

Les Igoudar au Maroc

-  **Greniers collectifs fortifiés**
Régions du Souss et de l'Anti-Atlas
-  **Banque communautaire**
Rôle similaire à nos banques modernes
-  **Système sécurisé**
Gestion des biens et des richesses

Fonctionnement des Igoudar

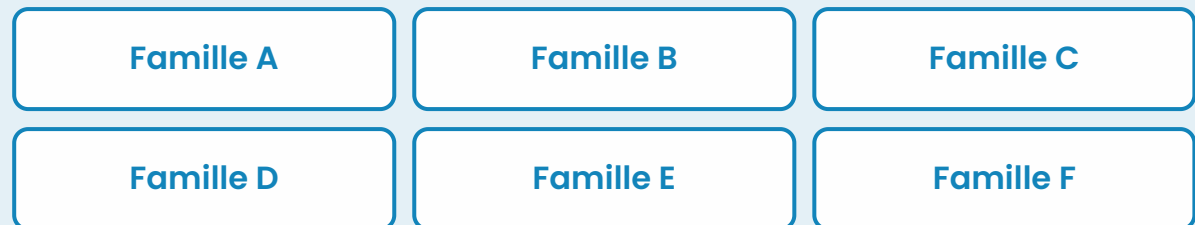
 **Par famille**
Cellule privée

 **Identification**
Symbole ou numéro

Biens stockés dans les Igoudar



Schéma simplifié d'un Igoudar



→ Chaque cellule = enregistrement sécurisé

1.2.1 Gestion traditionnelle : cas des Igoudar

⇔ Équivalences modernes

- Donnée → Quantité de céréales
- Base de données → Igoudar
- Enregistrement → Cellule
- Sécurité → Gardien + règles
- Administrateur → Amin (responsable)

⚠ Limites de la gestion traditionnelle

- ✗ Gestion manuelle
- ✗ Pas de mise à jour rapide
- ✗ Accès limité
- ✗ Dépendance à la mémoire humaine

Schéma simplifié d'un Igoudar



IG OUDAR

Famille A

Cellule #1

Famille B

Cellule #2

Famille C

Cellule #3

Famille D

Cellule #4

Famille E

Cellule #5

Famille F

Cellule #6

→ Chaque cellule = enregistrement sécurisé

1.2.2 Gestion informatisée par fichiers



Transition informatique

Avec l'informatique, les données ont été stockées dans des fichiers informatiques, permettant une gestion plus rapide et efficace.

Exemples de fichiers

 etudiants.txt

 notes.txt

 enseignants.txt

Schéma de gestion par fichiers

Application A



Fichier Étudiants

Données brutes

Application B



Fichier Notes

Données redondantes

Application C



Fichier Étudiants

Données redondantes

⚠ Redondance des données : même information stockée plusieurs fois

1.3 Problèmes de la gestion par fichiers



Redondance

Même information stockée plusieurs fois

- × Nom étudiant répété
- × Adresse dupliquée
- × Données dispersées



Incohérence

Informations contradictoires

Fichier 1 : Ahmed, 25 ans

Fichier 2 : Ahmed, 26 ans

? Quelle est la bonne information?



Dépendance

Programme et fichier couplés

- ⇔ Programme ↔ Structure
- <> Changer fichier = Changer programme
- ⚙ Maintenance difficile

Schéma récapitulatif



× Couplage fort = Problèmes majeurs

1.4 Définition d'une Base de Données



Définition

Ensemble structuré de données, stockées de manière permanente, permettant un accès **efficace**, **sécurisé** et **partagé**.

✓ Caractéristiques

- Centralisée
- Cohérente
- Persistante
- Sécurisée
- Partagée



Exemple concret

Base de données universitaire :



Étudiants



Enseignants



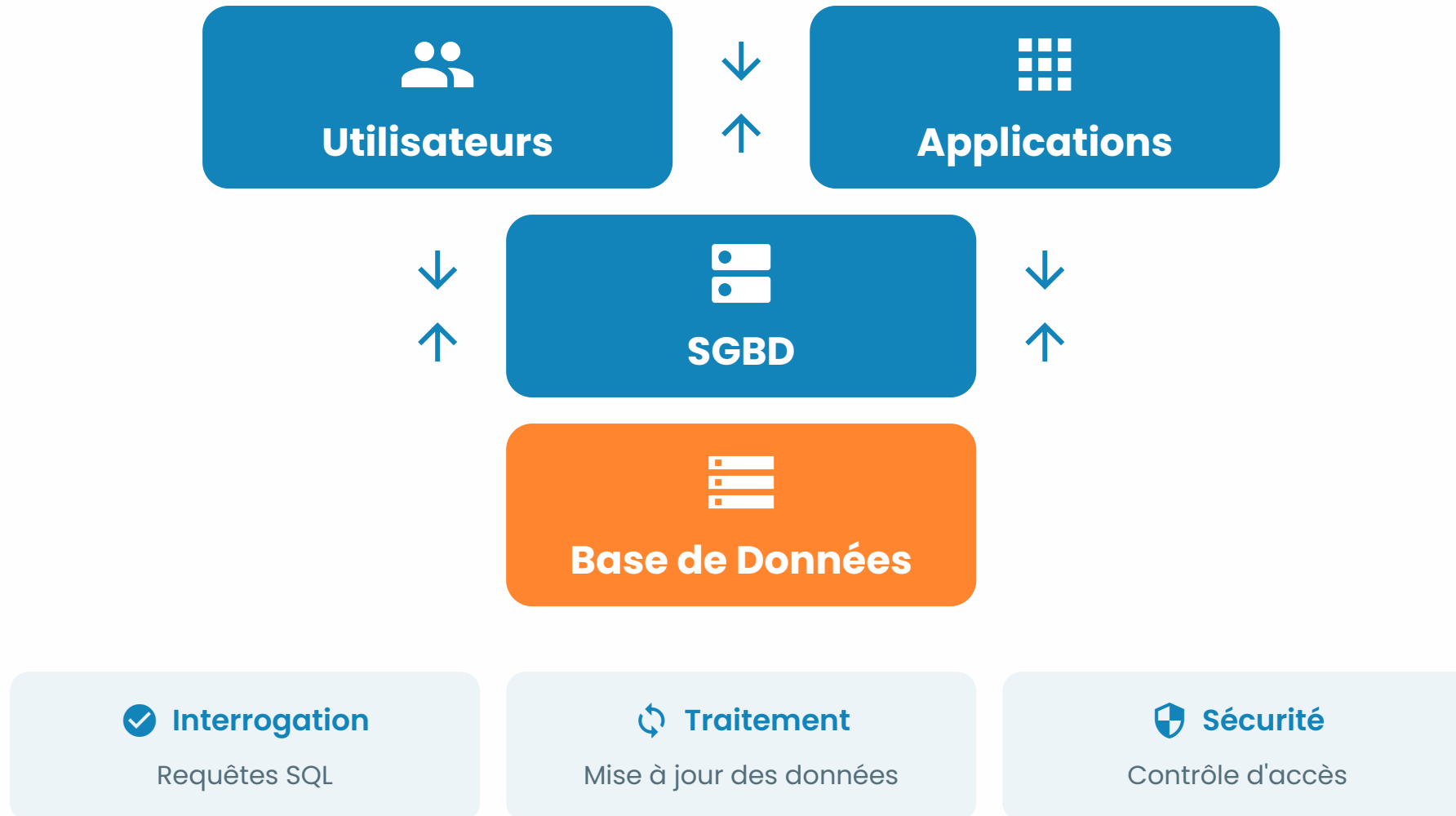
Modules



Notes

1.5 Système de Base de Données

Architecture du système



1.6 Objectifs des Bases de Données



Réduction de la redondance

Une donnée stockée une seule fois



Cohérence

Toute modification est visible partout



Indépendance des données

Les changements n'affectent pas les programmes



Sécurité

Mots de passe, droits d'accès, traçabilité

Schéma de l'indépendance des données



✓ Modifications possibles sans affecter les couches supérieures

1.7 Domaines d'application

Les bases de données sont omniprésentes



Banques

Comptes, transactions



Universités

Étudiants, notes



Hôpitaux

Patients, dossiers médicaux



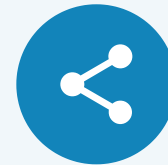
Administrations

Citoyens, registres



E-commerce

Clients, commandes, produits



Réseaux sociaux

Utilisateurs, publications

1.8 Acteurs d'un Système de Base de Données

Les principaux acteurs



Administrateur (DBA)

Sécurité

Sauvegarde

Performance



Concepteur

Modélisation

Schémas



Développeur

Applications

Requêtes SQL



Utilisateur final

Saisie

Consultation

1.9 Conclusion du Chapitre

De l'Igoudar aux bases de données modernes



L'objectif reste le même



Stocker



Protéger



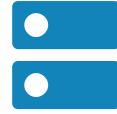
Exploiter

l'information de manière fiable

✓ **Essentielles**

✓ **Évolution**

✓ **Rôle central**



CHAPITRE 2

SYSTÈME DE GESTION DE BASES DE DONNÉES (SGBD)

Cours de Bases de Données



Gestion



Sécurité



Fonctionnalités

2.1 Introduction

Contexte

Avec le développement des systèmes informatiques, les organisations doivent gérer des volumes de données croissants.

Les défis



Grandes quantités

Volumes massifs de données



Multi-utilisateurs

Accès simultané aux données



Sécurité et fiabilité

Exigences élevées

La solution : SGBD

Une base de données seule ne suffit pas. Il faut un logiciel spécialisé pour :

- ✓ Organiser les données
- ✓ Contrôler l'accès
- ✓ Exploiter efficacement

→ **Système de Gestion de Bases de Données**

2.2 Définition d'un SGBD

Qu'est-ce qu'un SGBD ?

Un logiciel permettant de gérer les données de manière **efficace et sécurisée**

 Créer des bases de données

 Définir la structure

 Stocker de manière organisée

 Interroger et modifier

 Assurer sécurité et cohérence

Rôle d'intermédiaire



▮ Les utilisateurs n'accèdent jamais directement aux données physiques

2.3 Rôle du SGBD

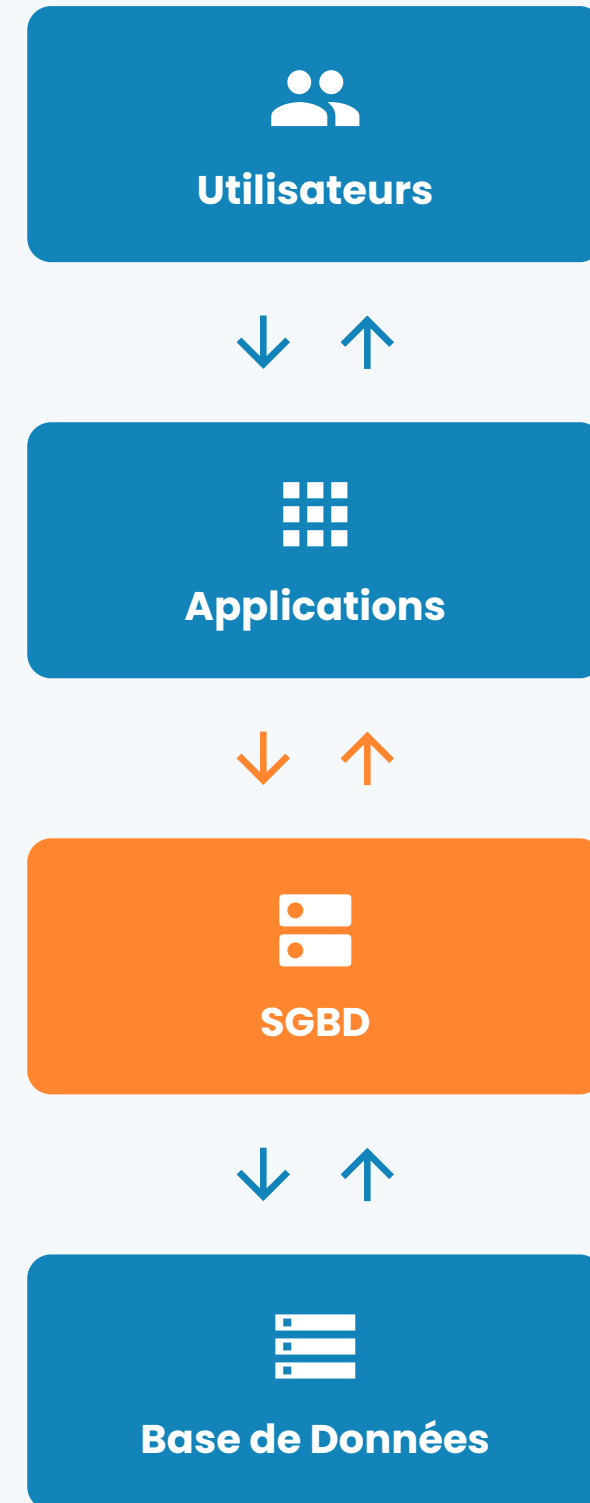
Rôle central

Le SGBD joue un rôle essentiel dans le système d'information en agissant comme **intermédiaire** entre les utilisateurs et les données.

Accès direct interdit

Les utilisateurs n'accèdent **jamais** directement aux données physiques. Toutes les opérations passent par le SGBD.

Architecture du système



2.4 Fonctions principales d'un SGBD - Partie 1



Définition des données

Permet de définir la structure de la base de données

- Les tables
- Les attributs
- Les types de données
- Les contraintes d'intégrité

<> Langage de Définition de Données (LDD)

CREATE

ALTER

DROP

▮ Le LDD permet de créer, modifier et supprimer les structures de la base de données



Manipulation des données

Permet de gérer le contenu de la base de données



Insertion

Ajouter de nouvelles données



Mise à jour

Modifier des données existantes



Suppression

Retirer des données de la base

<> Langage de Manipulation de Données (LMD)

INSERT

UPDATE

DELETE

▮ Le LMD permet de modifier le contenu des tables

2.4 Fonctions principales d'un SGBD - Partie 2



Interrogation

Permet de rechercher et analyser les données

- Rechercher
- Filtrer
- Trier
- Regrouper

Langage SQL

SELECT



Sécurité

Protège les données contre les accès non autorisés



Authentification

Vérification des utilisateurs



Droits & Privilèges

Contrôle d'accès



Protection

Données sensibles



Concurrence

Gère les accès simultanés sans conflits

Plusieurs utilisateurs peuvent accéder simultanément

✓ Sans incohérence



Transactions

Opérations atomiques



Verrouillage

Mécanismes de protection



Point clé

Ces fonctions permettent de garantir l'intégrité, la confidentialité et la disponibilité des données

2.4 Fonctions principales d'un SGBD - Partie 3



Sauvegarde et restauration

Le SGBD assure la protection continue des données grâce à des mécanismes de sauvegarde automatique et de restauration.



Sauvegardes automatiques

Copies régulières de la base de données



Restauration

Récupération après panne matérielle ou logicielle



Fréquence des sauvegardes

- Quotidiennes : données transactionnelles
- Hebdomadaires : configurations système
- Mensuelles : archives historiques



Avantages clés



Protection

Contre la perte



Continuité

Service garanti



Historique

Versions disponibles



Sérénité

Esprit tranquille

2.5.1 Architecture à un niveau

Concept

Tous les composants résident sur la **même machine**

Exemple

Base de données locale sur un ordinateur personnel

Structure

Utilisateur + Application + Base de Données

Avantages



Simplicité

Installation et configuration faciles

Inconvénients



Sécurité limitée

Données exposées localement



Pas adaptée au multi-utilisateur

Accès concurrentiel impossible

2.5.2 Architecture à deux niveaux (Client/Serveur)

Architecture Client/Serveur

Séparation entre l'interface utilisateur et le système de gestion de données

Exemple

Application web avec serveur MySQL

Structure de l'architecture



CLIENT

Interface utilisateur



SERVEUR

SGBD + Base de données



Meilleure performance

Répartition de la charge



Sécurité améliorée

Contrôle d'accès centralisé

2.5.3 Architecture à trois niveaux (ANSI/SPARC)

🏰 Architecture ANSI/SPARC

Séparation en trois niveaux indépendants pour une gestion flexible des données

★ **Avantage clé**

Permet l'**indépendance des données**

Les trois niveaux



NIVEAU EXTERNE

Vues utilisateurs



NIVEAU CONCEPTUEL

Schéma global



NIVEAU INTERNE

Stockage physique

2.6 Indépendance des données



Indépendance physique

Modification du stockage **sans changer les programmes**

< > Exemples



Ajout d'index

Optimisation des performances



Changement de disque dur

Nouvel emplacement de stockage



Indépendance logique

Modification du schéma logique **sans affecter les applications**

< > Exemples



Ajout d'un attribut

Extension de la structure



Création d'une nouvelle table

Extension du schéma

2.7.1-2 Types de SGBD



Hiérarchiques



Organisation en **arbre**



Relation **parent-enfant**

× Limite

Peu flexible, navigation complexe



Réseaux



Relations **multiples**



Structure plus **souple** que le hiérarchique



Avantage

Navigation plus flexible entre les données

2.7.3 Types de SGBD - Relationnels



Relationnels



Basés sur des **tables**



Utilisation du langage **SQL**



Les plus utilisés aujourd'hui



Popularité

Standard de l'industrie



Exemples de SGBD relationnels



MySQL

Open source, très populaire



Oracle

Entreprise, haute performance



PostgreSQL

Open source, avancé



SQL Server

Microsoft, entreprise

2.7.4-5 Types de SGBD



Orientés objet

- ✓ Stockage d'**objets complexes**
- <> Proche de la **programmation orientée objet**



Avantage

Intégration naturelle avec le code applicatif



Objet-relationnels

- + Combinaison du **relationnel** et de **l'objet**
- ◇ Flexibilité du modèle objet
- ▮ Performance du modèle relationnel



Avantage

Meilleur des deux mondes

2.8 Exemple de SGBD relationnel



Table ETUDIANT

Colonnes

- id_etudiant (Clé primaire)
- nom
- age

i Caractéristiques

Structure de table relationnelle standard

Données d'exemple

id_etudiant	nom	age
1	Ahmed	22
2	Sara	21
3	Youssef	23

Nombre
d'enregistrements
3

Nombre de colonnes
3

Type de table
Relationnel

2.9 Avantages des SGBD



Réduction de la redondance

Chaque donnée stockée une seule fois, éliminant les duplications inutiles



Cohérence des données

Toute modification est visible partout, garantissant l'intégrité



Sécurité renforcée

Contrôle d'accès, authentification et protection des données sensibles



Accès multi-utilisateurs

Plusieurs utilisateurs peuvent travailler simultanément sans conflits



Maintenance facilitée

Gestion centralisée et outils d'administration puissants

2.10 Inconvénients des SGBD



Coût élevé

Matériel

Serveurs puissants et infrastructure spécialisée

Logiciel

Licences coûteuses et frais d'abonnement



Complexité



Mise en œuvre complexe



Formation nécessaire



Configuration avancée



Intégration difficile



Administrateur



Nécessite un DBA

Administrateur de Base de Données qualifié



Compétences rares

Expertise technique avancée requise



Ressources humaines

Salaires élevés et formation continue

2.11 Acteurs d'un SGBD



Administrateur (DBA)

-  Gestion de la sécurité
-  Sauvegarde et restauration
-  Optimisation des performances



Concepteur

-  Modélisation des données
-  Conception des schémas
-  Définition des contraintes



Développeur

-  Développement d'applications
-  Écriture de requêtes SQL
-  Intégration avec les applications



Utilisateur final

-  Consultation des données
-  Saisie des informations
-  Recherche et exploitation

2.12 Conclusion du Chapitre



Le SGBD : Cœur du système

Le Système de Gestion de Bases de Données est le composant central qui permet une gestion efficace et sécurisée des données.



Intermédiaire essentiel entre utilisateurs et données

✓= Garanties du SGBD



Fiabilité

Données correctes et cohérentes



Sécurité

Contrôle d'accès et protection



Cohérence

Intégrité des données



Exploitation efficace

Performance et accessibilité



CHAPITRE 3

MODÉLISATION CONCEPTUELLE DES DONNÉES

Modèle Entité–Association



Base de Données



Modèle EA



Conception

3.1 Introduction à la modélisation des données

Avant de créer une base de données...

Il est indispensable de bien comprendre le système réel que l'on veut représenter.

- Analyser la réalité
- Organiser les informations
- Éviter les erreurs de conception

Modèle Entité–Association (EA)

- ✓ Représentation **conceptuelle** de la base de données
- ✓ Indépendante de tout **SGBD**
- ✓ Approche méthodique et structurée

3.2 Objectifs de la modélisation conceptuelle



Décrire les données

Comprendre et documenter la structure des informations du système



Identifier les relations

Établir les connexions et liens entre différentes entités



Faciliter la communication

Créer un langage commun entre utilisateurs et informaticiens



Préparer le modèle relationnel

Base solide pour la transition vers la base de données



Une modélisation claire garantit une base de données efficace et maintenable

3.3 Notions fondamentales du modèle Entité–Association

Le modèle EA repose sur **3 concepts essentiels**



Entité

Objet du monde réel,
identifiable de manière
unique

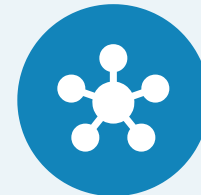
Personne, Objet, Concept



Attribut

Propriété qui décrit une
entité

Caractéristiques



Association

Lien logique entre deux
ou plusieurs entités

Relations

3.4 Entité - Définition et exemples



Définition

Objet du monde réel, identifiable de manière unique

Types d'entités



Personne

Étudiant, Enseignant



Objet

Livre, Salle



Concept

Module, Compte

Exemples d'entités

Dans un système universitaire

Étudiant

Enseignant

Module

Département

Schéma textuel d'une entité

[ETUDIANT]

3.5 Attribut - Définition et exemples



Définition

Propriété qui décrit une entité

Exemples d'attributs

Pour l'entité **ETUDIANT** :

- id_etudiant
- nom
- prénom
- date_naissance
- filière

Schéma textuel

[ETUDIANT]

id_etudiant

nom

prénom

date_naissance

filière

 Les attributs décrivent les caractéristiques d'une entité

3.6 Identifiant (clé)



Définition

Attribut (ou groupe d'attributs) permettant d'identifier de manière unique chaque occurrence d'une entité

Exemple

Dans l'entité **ETUDIANT** :

- **id_etudiant** est l'identifiant
- Chaque étudiant a un **ID unique**

Notation dans le schéma textuel

[ETUDIANT]

id_etudiant

nom

prénom

date_naissance

filière

Le symbole

Indique l'attribut identifiant dans le schéma textuel

3.7 Types d'attributs



Attribut simple

Non décomposable

→ Âge



Attribut composé

Peut être décomposé

→ Nom_complet → nom + prénom



Attribut multivalué

Plusieurs valeurs possibles

→ Téléphone



Attribut dérivé

Calculé à partir d'autres attributs

→ Âge (depuis date_naissance)

3.8 Association - Définition et exemple



Définition

Lien logique entre deux ou plusieurs entités

Exemple concret

- Un **étudiant** s'inscrit à un **module**
- Association nommée **S'INSCRIT**
- Lie les entités **ETUDIANT** et **MODULE**

Schéma textuel

[ETUDIANT
]

S'INSCRIT

[MODULE]



L'association **S'INSCRIT** représente le lien entre un étudiant et un module

3.9 Degré d'une association



Binaire

Association entre **2 entités**

Le plus courant



Exemple : Étudiant → Module



Ternaire

Association entre **3 entités**

→ Plus complexe

→ Moins fréquent



N-aire

Association entre **plus de 3 entités**

→ Très complexe

→ Rare en pratique



La majorité des associations dans la modélisation conceptuelle sont **binaires**

3.10 Cardinalités



Définition

Nombre **minimum** et **maximum** d'occurrences d'une entité pouvant participer à une association

Notation

(min, max)



Permet de caractériser les contraintes de participation

Les 4 types de cardinalités

(0, 1)

Optionnelle et unique

Zéro ou une occurrence

(1, 1)

Obligatoire et unique

Exactement une occurrence

(0, N)

Optionnelle et multiple

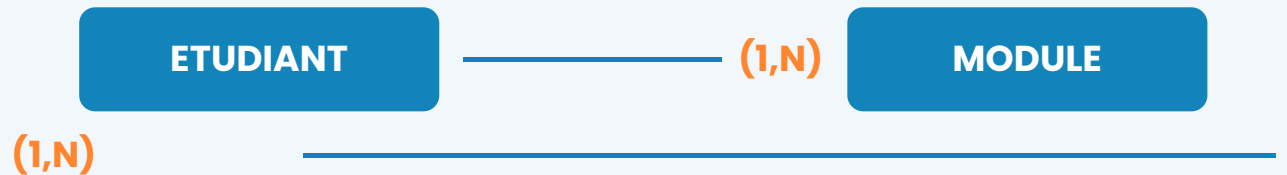
Zéro ou plusieurs occurrences

(1, N)

Obligatoire et multiple

Au moins une occurrence

Exemple concret



Un étudiant peut s'inscrire à plusieurs modules, un module peut avoir plusieurs étudiants

3.11 Attributs d'association



Définition

Une association peut avoir ses propres attributs, distincts des entités qu'elle relie

Pourquoi ?

- ✓ L'information dépend des deux entités
- ✓ Existe uniquement dans le contexte de la relation

Exemple concret

L'association **S'INSCRIT** possède ses propres attributs

ETUDIANT

S'INSCRIT

MODULE

- date_inscription
- note



Attribut

date_inscription



Attribut

note

3.12 Contraintes d'intégrité

Les 3 types de contraintes



Contraintes d'unicité

Chaque identifiant doit être unique



Contraintes de participation

Obligatoire ou facultative selon les cardinalités



Contraintes de cohérence

Règles métier à respecter



Exemple concret

Dans le système universitaire, certaines règles métier doivent être appliquées



Un étudiant ne peut pas avoir une note sans être inscrit



L'attribut **note** dépend de l'association **S'INSCRIT**

3.13 Héritage (Spécialisation / Généralisation)



Définition

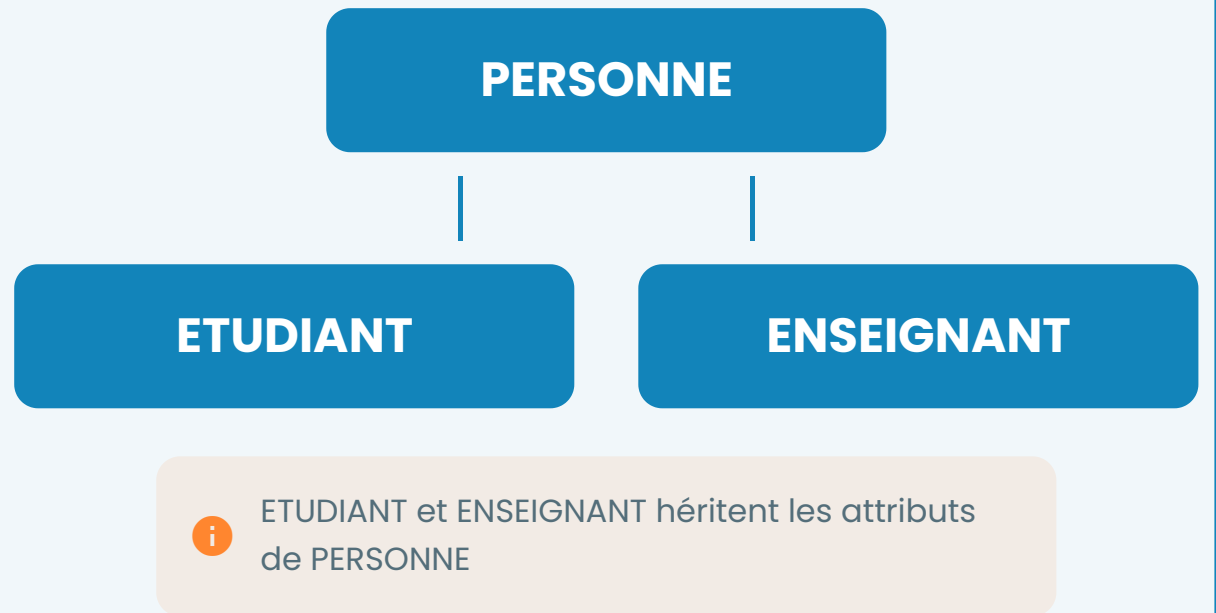
Permet de définir une **entité générale** et des **entités spécialisées**

Concepts clés

- ↑ **Généralisation**
Entité mère plus abstraite
- ↓ **Spécialisation**
Entités filles plus spécifiques
- ↔ **Héritage**
Attributs communs transférés

Exemple concret

Structure hiérarchique dans un système universitaire



3.14 Exemple complet de modélisation

Système universitaire

Entités

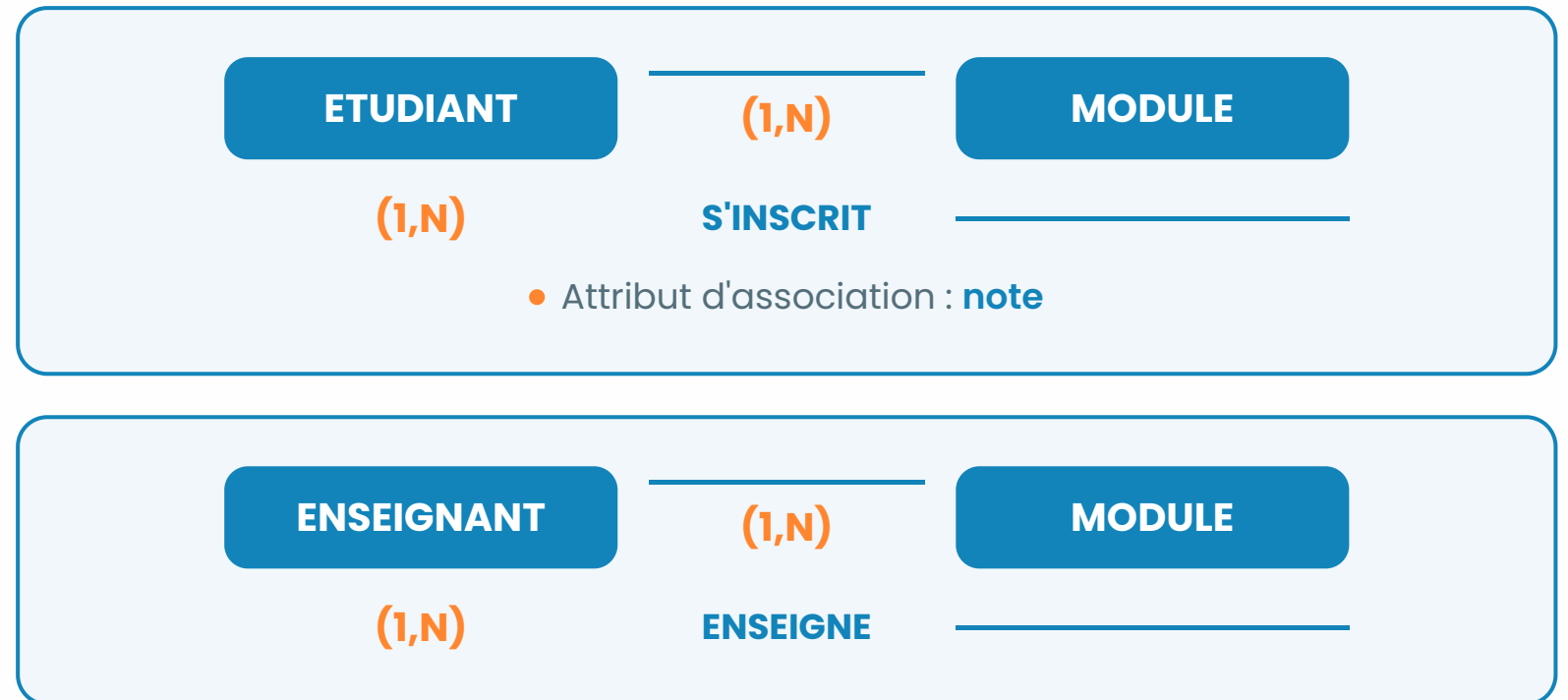
- ETUDIANT
- MODULE
- ENSEIGNANT

Associations

- S'INSCRIT
- ENSEIGNE



Un étudiant peut s'inscrire à plusieurs modules



3.15 Règles de bonne modélisation

✂= 5 règles essentielles



Éviter la redondance

Chaque information doit être stockée à un seul endroit



Utiliser des noms clairs

Noms significatifs et faciles à comprendre



Identifiants stables

Choisir des clés qui ne changent pas dans le temps



Respecter les règles métier

Intégrer les contraintes et spécificités du domaine



Valider avec les utilisateurs

Assurer la conformité aux besoins et attentes des utilisateurs finaux



Conclusion du chapitre

Le modèle **Entité-Association** est une étape fondamentale dans la conception d'une base de données.



Représentation claire

Du système réel



Indépendance

Vis-à-vis des SGBD



Base solide

Pour le modèle
relationnel



CHAPITRE 4

LE MODÈLE RELATIONNEL

Cours de Bases de Données



Tables



Clés



Relations

4.1 Introduction

Modèle Entité-Association

Modélisation conceptuelle des données

Passage obligatoire

Vers un modèle logique exploitable

Modèle Relationnel

Structure directement utilisable par les SGBD



Point clé

Le modèle relationnel constitue la base théorique des bases de données relationnelles et du langage SQL.



SGBD Relationnels

MySQL, Oracle,
PostgreSQL...



Langage SQL

Standard international

4.2 Concepts fondamentaux du modèle relationnel - Partie 1



Relation

Une relation correspond à une **table** dans une base de données.

☰ Définie par :

- Un nom
- Un ensemble d'attributs
- Un ensemble de tuples

Exemple :

ETUDIANT(id_etudiant, nom, prenom, filiere)



Attribut

Un attribut représente une **colonne** d'une table.

⚙ Défini sur un domaine (type de données)

Exemples :

id_etudiant → entier

nom → chaîne de caractères

i Chaque colonne correspond à un attribut de la relation

4.2 Concepts fondamentaux du modèle relationnel - Partie 2

Domaine

Ensemble des valeurs possibles qu'un attribut peut prendre

Correspondance Attribut - Domaine

Attribut	Domaine
âge	Entiers positifs
note	[0,20]
date	Format date

Tuple

Un tuple correspond à une **ligne** de la table

ETUDIANT			
id	nom	prenom	filiere
1	Ahmed	Ali	Informatique
2	Sara	Ben	Mathématiques



Chaque tuple représente un enregistrement complet dans la relation

4.3 Schéma relationnel



Définition

Un schéma relationnel décrit la structure complète d'une base de données relationnelle.



Comprend :

- Les relations (tables)
- Les attributs (colonnes)
- Les clés (primaires et étrangères)
- Les liens entre relations



Notation



Exemple de schéma :

```
ETUDIANT(#id_etudiant, nom, prenom,  
         filiere)
```



Signification du



Clé primaire

Identifie de manière unique chaque tuple



Ne peut pas être NULL

4.4 Les clés dans le modèle relationnel - Partie 1



Clé primaire

Une clé primaire est un attribut (ou groupe d'attributs) qui :

- Identifie de manière **unique** chaque tuple
- Ne peut pas être **NULL**

<> Notation :

Attribut précédé de **#** dans le schéma

Exemple :

```
ETUDIANT(#id_etudiant, nom, prenom)
```



Clé candidate

Une clé candidate est un attribut pouvant servir de clé primaire.

📄 Caractéristiques :

- Identifie de manière unique
- Peut être choisi comme clé primaire

☰ Exemples :



id_etudiant



numéro_carte

4.4 Les clés dans le modèle relationnel - Partie 2

Clé étrangère

Une clé étrangère est un attribut qui **référence la clé primaire** d'une autre table.

Notation :

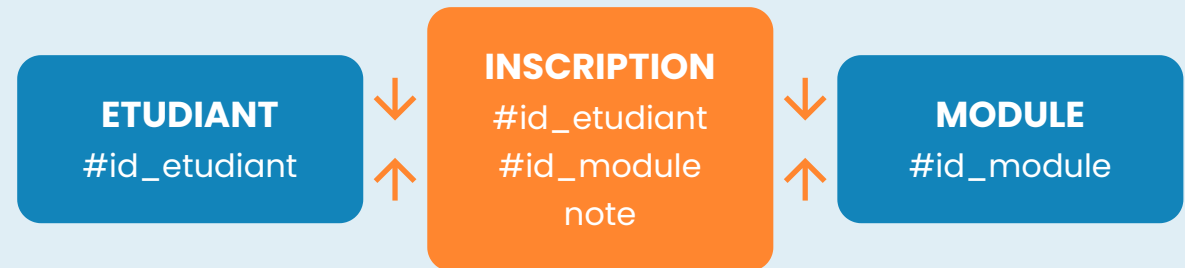
Clé étrangère aussi indiquée par **#** dans le schéma

Mais référence une clé primaire d'une autre table

Exemple :

```
INSCRIPTION(#id_etudiant,  
            #id_module, note)
```

Schéma illustratif



 Les clés étrangères établissent les **liens entre les tables**

4.5 Contraintes d'intégrité



Intégrité de domaine

Les valeurs doivent être conformes au domaine



Intégrité d'entité

Clé primaire unique et non nulle



Intégrité référentielle

Une clé étrangère doit référencer une clé existante



Schéma d'intégrité référentielle

ETUDIANT

#id_etudiant
nom



INSCRIPTION

id_etudiant
#id_module
note



L'intégrité référentielle garantit la **validité des liens** entre les tables

4.6 Passage du modèle EA au modèle relationnel - Partie 1



Transformation des entités

Chaque entité devient une table dans le modèle relationnel.

- Identifiant → Clé primaire
- Attributs → Colonnes
- Nom entité → Nom table



Exemple de transformation

Modèle EA

[
ETUDIANT
]



Modèle Relationnel

ETUDIANT(#id_etudiant,
nom, prenom)



Le passage est **systematique** et direct

4.6 Passage du modèle EA au modèle relationnel - Partie 2



Association 1,N

- La clé primaire du côté 1 devient clé étrangère du côté N

<> Exemple :

DEPARTEMENT

#id_dep
nom

ETUDIANT

#id_etudiant
nom
#id_dep



Association N,N

- Création d'une nouvelle table associative

<> Exemple :

INSCRIPTION

#id_etudiant (clé étrangère → ETUDIANT)
#id_module (clé étrangère → MODULE)
note

i La table associative contient les clés étrangères des deux tables liées

4.7 Anomalies de mise à jour & 4.8 Introduction à la normalisation



Anomalie d'insertion

Impossible d'insérer une donnée sans une autre



Anomalie de suppression

Suppression involontaire d'informations importantes



Anomalie de modification

Modification multiple d'une même donnée



La normalisation

La normalisation permet d'éliminer les redondances et d'éviter les anomalies de mise à jour.

- Éliminer les redondances
- Éviter les anomalies d'insertion, suppression, modification
- Améliorer la cohérence des données

Formes normales

Elle repose sur les formes normales (1FN, 2FN, 3FN...)



La normalisation sera détaillée dans le chapitre suivant

4.9-4.12 Exemple complet et Conclusion

Système universitaire

ETUDIANT

#id_etudiant, nom, filiere



INSCRIPTION

#id_etudiant, #id_module, note



MODULE

#id_module, libelle, credit

Avantages

- Simplicité
- Fondement mathématique solide
- Langage SQL standardisé
- Indépendance des applications

Limites

Difficulté à gérer des objets complexes

Conclusion

Le modèle relationnel constitue le pont entre la modélisation conceptuelle et l'implémentation physique.

- Organisation rigoureuse des données
- Exploitation efficace par les SGBD
- Mise en œuvre du langage SQL



CHAPITRE 5

NORMALISATION DES BASES DE DONNÉES

Cours de Bases de Données

✓= Qualité

✚ Structure

✓ Fiabilité

5.1 Introduction

Mauvaise organisation

Une mauvaise organisation des tables peut entraîner :



Des redondances



Des incohérences



Des anomalies de mise à jour

La solution : Normalisation

Une démarche méthodique pour structurer correctement les relations

La normalisation permet



Structurer correctement les relations

Organisation logique et cohérente des tables



Éliminer les redondances

Une seule source de vérité pour chaque donnée



Garantir la cohérence

Données fiables et maintenues à jour

5.2 Objectifs de la normalisation

La normalisation vise à

Concevoir des bases de données optimales en éliminant les redondances et les anomalies de mise à jour.



Réduire la duplication

Moins de redondance, meilleure efficacité



Éviter les anomalies

Insérer, supprimer, modifier sans erreur



Améliorer la maintenance

Faciliter l'évolution de la base



Garantir l'intégrité

Données cohérentes et fiables

5.3 Dépendances fonctionnelles

Σ Définition

Une dépendance fonctionnelle (DF) existe lorsqu'un attribut dépend fonctionnellement d'un autre.

mp

<> Notation

A → B

La valeur de A détermine de manière unique la valeur de B

Exemple concret

Dans la relation :

ETUDIANT(#id_etudiant, nom, filiere)

On a les dépendances suivantes :

→ **id_etudiant → nom**

→ **id_etudiant → filiere**

5.4 Anomalies dues à la non-normalisation



Une mauvaise organisation des tables entraîne des anomalies lors des opérations CRUD



Anomalie d'insertion

Impossible d'insérer certaines données

Sans en ajouter d'autres inutilement



Anomalie de suppression

La suppression d'une ligne entraîne

La perte d'informations importantes



Anomalie de modification

Une modification doit être

Répétée plusieurs fois



Solution : La normalisation permet d'éliminer ces anomalies en structurant correctement les données

5.5 Première Forme Normale (1FN)

✖ Définition

Une relation est en 1FN si :

- Tous les attributs sont atomiques
- Aucune valeur multivaluée
- Aucune valeur composée

⚠ Les valeurs ne doivent pas être divisibles

✖ Exemple non 1FN

ETUDIANT(#id, nom, telephones)
telephones = 0612, 0723 (valeur multivaluée)

✓ Passage en 1FN

ETUDIANT(#id, nom)

TELEPHONE(#id, telephone)

i Création de deux tables distinctes

5.6 Deuxième Forme Normale (2FN)

✖ Définition

Une relation est en 2FN si :

- Elle est en 1FN
- Aucun attribut non clé ne dépend d'une partie de la clé primaire

i Pas de dépendance partielle

✖ Exemple non 2FN

INSCRIPTION(#id_etudiant, #id_module, nom_etudiant)

id_etudiant $\rightarrow$ nom_etudiant (dépendance partielle)

⚠ L'attribut nom_etudiant ne dépend que de id_etudiant

✓ Correction

ETUDIANT(#id_etudiant, nom_etudiant)

INSCRIPTION(#id_etudiant, #id_module)

⚙ Suppression de la dépendance partielle

5.7 Troisième Forme Normale (3FN)

✖ Définition

Une relation est en 3FN si :

- Elle est en 2FN
- Aucun attribut non clé ne dépend d'un autre attribut non clé

i Pas de dépendance transitive

✗ Exemple non 3FN

ETUDIANT(#id, nom, id_dep, nom_dep)

$id_dep \rightarrow nom_dep$ (dépendance transitive)

! nom_dep dépend de id_dep, pas de la clé

✓ Correction

ETUDIANT(#id, nom, id_dep)

DEPARTEMENT(#id_dep, nom_dep)

✓ Suppression de la dépendance transitive

5.8 Forme Normale de Boyce-Codd (BCNF)

≡ Définition

Une relation est en BCNF si :

Pour toute dépendance fonctionnelle $A \rightarrow B$, A est une clé candidate

 Dépendances basées uniquement sur les clés candidates

Remarque

- La BCNF est plus stricte que la 3FN
- Parfois plus complexe à mettre en œuvre

→ BCNF vs 3FN

Dépendance transitive

$A \rightarrow B$ et $B \rightarrow C$, mais A ne dépend pas de C

 **Aucune dépendance partielle ou transitive**
Toutes les dépendances $A \rightarrow B$ ont A comme clé candidate

 BCNF assure une meilleure intégrité des données

5.9 Résumé des formes normales

1 1FN

- ✓ Attributs atomiques

2 2FN

- ✓ Pas de dépendance partielle

3 3FN

- ✓ Pas de dépendance transitive

B BCNF

- ✓ Dépendance basée sur clés

↗ Progression

Chaque forme normale corrige des anomalies spécifiques et améliore la qualité de la base de données

★ Objectif

Éliminer les redondances et garantir l'intégrité des données

5.10 Exemple complet de normalisation



Relation initiale

```
COMMANDE(#id_commande, date,  
id_client, nom_client,  
adresse_client)
```

1 Étape 1 : 1FN

✓ Déjà atomique

2 Étape 2 : 2FN

✓ Clé simple → OK

3 Étape 3 : 3FN

```
CLIENT(#id_client, nom_client, adresse_client)
```

```
COMMANDE(#id_commande, date, id_client)
```

5.11-5.12 Avantages et Limites

Avantages

-  **Réduction des redondances**
Moins de duplication des données
-  **Cohérence accrue**
Données plus fiables
-  **Meilleure maintenance**
Facilite les mises à jour
-  **Économie d'espace**
Stockage optimisé

Limites

-  **Augmentation du nombre de tables**
Plus de relations à gérer
-  **Requêtes plus complexes**
Jointures multiples
-  **Besoin de dénormalisation**
Pour certaines performances

5.13 Conclusion du Chapitre

Étape essentielle

La normalisation est indispensable pour garantir la qualité et la fiabilité d'une base de données relationnelle

Éviter les anomalies

Insérer, supprimer, modifier sans erreur

Structurer efficacement

Organisation logique des données

Garantir l'intégrité

Données cohérentes et fiables

 **La normalisation constitue la base solide pour une base de données relationnelle de qualité**

— CHAPITRE 6

INTRODUCTION AU LANGAGE SQL

Structured Query Language - Le langage standard des bases de données relationnelles

☰ Bases de données

<> Manipulation de données

🔍 Interrogation SQL

6.1 Introduction

✚ Pourquoi SQL ?

Après la **conception logique** (modèle relationnel et normalisation), un langage standard est nécessaire pour :

- Créer les bases de données
- Manipuler les données
- Interroger les informations

📄 Définition

SQL est un langage **déclaratif** : on décrit **ce que l'on veut obtenir** et non comment l'obtenir.

☰ SGBD Relationnels



MySQL



Oracle



PostgreSQL



SQL Server

| 6.2 Présentation du langage SQL

★ Caractéristiques principales



Langage Standard

Norme **ANSI / ISO** reconnue mondialement



Indépendant des SGBD

Compatible avec la majorité des SGBD (avec spécificités)



Simple à Apprendre

Syntaxe intuitive et compréhensible



Puissant

Manipulation efficace de **grandes quantités de données**

6.3 Catégories du langage SQL



LDD / DDL

Langage de Définition de Données



LMD / DML

Langage de Manipulation de Données



LRD / DQL

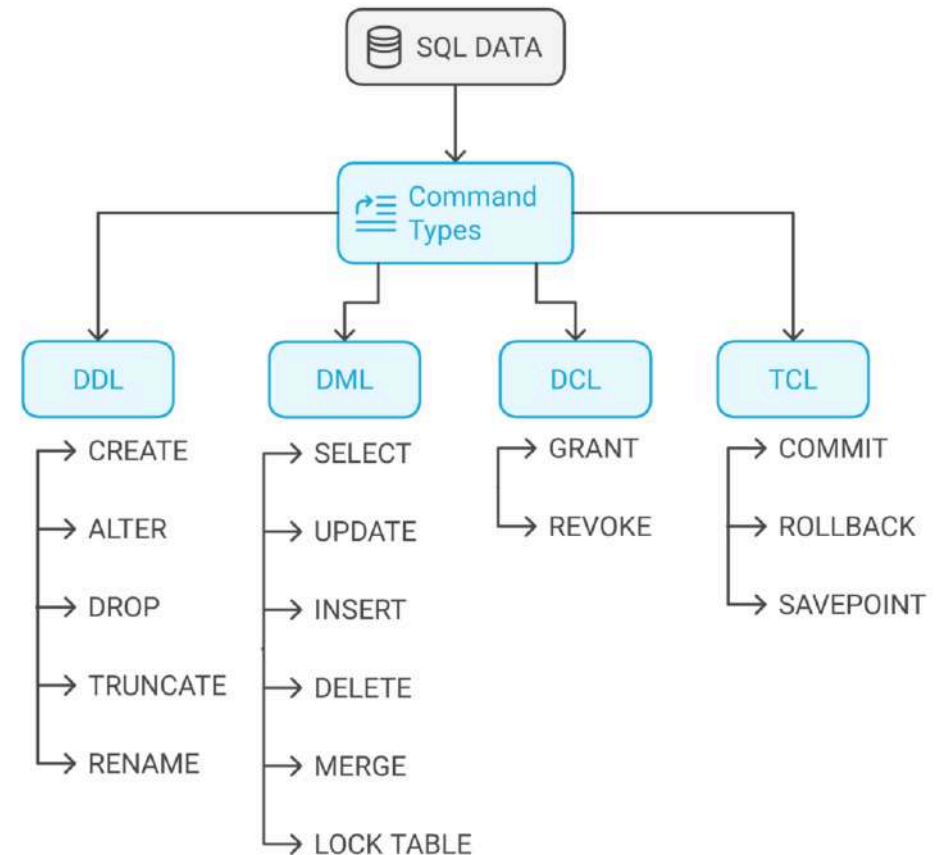
Langage de Requête de Données



LCL / TCL

Langage de Contrôle des Transactions

Diagramme des commandes SQL



| 6.3.1 & 6.3.2 LDD et LMD



LDD

Langage de Définition de Données



CREATE Créer des objets



ALTER Modifier des objets



DROP Supprimer des objets



TRUNCATE Vider une table

<> Exemple

```
CREATE TABLE ETUDIANT (  
  id_etudiant INT,  
  nom VARCHAR(50),  
  filiere VARCHAR(30)  
);
```



LMD

Langage de Manipulation de Données



INSERT Ajouter des données



UPDATE Modifier des données



DELETE Supprimer des données

<> Exemples

```
INSERT INTO ETUDIANT  
VALUES (1, 'Ahmed', 'Informatique');
```

```
UPDATE ETUDIANT  
SET filiere='Mathématiques'  
WHERE id_etudiant=1;
```

```
DELETE FROM ETUDIANT  
WHERE id_etudiant=1;
```

| 6.3.3 & 6.3.4 LRD et LCL/TCL



LRD

Langage de Requête de Données



SELECT Interroger les données

<> Exemple

```
SELECT nom, filiere  
FROM ETUDIANT;
```

<> Avec filtre

```
SELECT nom  
FROM ETUDIANT  
WHERE filiere = 'Informatique';
```



LCL / TCL

Contrôle des Transactions



COMMIT Valider transaction



ROLLBACK Annuler transaction



SAVEPOINT Point de sauvegarde

<> Exemple

```
BEGIN TRANSACTION;  
  
UPDATE ETUDIANT  
SET filiere='Mathématiques'  
WHERE id_etudiant=1;  
  
COMMIT;
```

6.4 Structure d'une requête SQL

Syntaxe de base

Structure standard

 **SELECT** attributs

 **FROM** table

 **WHERE** condition

Exemple concret

Requête pratique

Requête SQL

```
SELECT nom  
FROM ETUDIANT  
WHERE filiere = 'Informatique';
```

Explication

- Sélectionne les **noms**
- De la table **ETUDIANT**
- Pour ceux en filière **Informatique**

| 6.5 Types de données & 6.6 Contraintes

Types de données

Numériques

- INT
- FLOAT
- DECIMAL

Caractères

- CHAR
- VARCHAR
- TEXT

Date/Heure

- DATE
- TIME
- DATETIME

Contraintes d'intégrité



Exemple complet

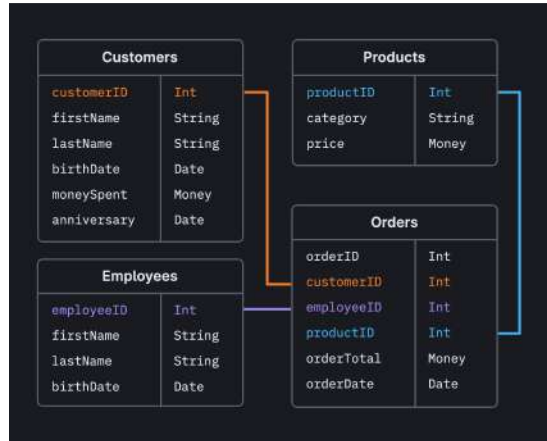
Table ETUDIANT

```
CREATE TABLE ETUDIANT (  
  id_etudiant INT PRIMARY KEY,  
  nom VARCHAR(50) NOT NULL,  
  age INT CHECK (age >= 18)  
);
```

-  **PRIMARY KEY** : identifiant unique
- NOT NULL** : valeur obligatoire
- CHECK** : impose condition

6.7 Exemple de base & 6.8 Schéma d'exécution

Exemple de base de données



<> Création des tables

```
CREATE TABLE MODULE (  
  id_module INT PRIMARY KEY,  
  libelle VARCHAR(50)  
);  
  
CREATE TABLE INSCRIPTION (  
  id_etudiant INT,  
  id_module INT,  
  note FLOAT,  
  PRIMARY KEY (id_etudiant, id_module),  
  FOREIGN KEY (id_etudiant) REFERENCES ETUDIANT,  
  FOREIGN KEY (id_module) REFERENCES MODULE  
);
```

Schéma d'exécution SQL



Requête SQL

SELECT, FROM, WHERE...



Analyse

Validation syntaxique



Optimisation

Plan d'exécution optimal



Exécution



Résultat

| 6.9 Avantages, Limites et Bonnes pratiques



Avantages



Simplicité

Facile à apprendre et utiliser



Standardisation

Norme ANSI/ISO reconnue



Portabilité

Compatible entre SGBD



Puissance

Gère de grandes quantités



Limites



Différences SGBD

Variations entre systèmes



Traitements complexes

Limité pour logique avancée



Extensions propriétaires

Fonctions spécifiques au SGBD



Bonnes pratiques



Noms clairs

Descriptifs et cohérents



Respecter contraintes

Intégrité des données



Éviter SELECT *

Préciser les colonnes



Commenter

Documenter le code

| 6.12 Conclusion du Chapitre

SQL : Le Langage Fondamental

Le SQL est le standard incontournable des bases de données relationnelles



Créer

Définir la structure des
bases de données



Manipuler

Gérer les données avec
précision



Interroger

Extraire l'information
efficacement

✓ **Maîtriser SQL est essentiel pour tout travail avec les bases de données relationnelles**



Chapitre 7 : SQL – Langage de Définition de Données (DDL)

Data Definition Language



CREATE



ALTER



DROP

SQL

Base de données relationnelle • Structure et Schéma

7.1 Introduction



Qu'est-ce que le DDL ?

Le **Langage de Définition de Données** (DDL – Data Definition Language) est une partie essentielle du SQL. Il permet de **définir, modifier et supprimer la structure des bases de données**.



Le DDL agit sur la **structure (schéma)** des données, et **non sur leur contenu**.



Définir

Créer bases de données et tables



Modifier

Changer la structure existante



Supprimer

Éliminer structures

7.2 Rôle du DDL dans une base de données

SQL

DDL



Structure

- ✓ Créer bases/tables
- ✓ Définir types
- ✓ Contraintes d'intégrité
- ✓ Modifier/Supprimer schéma

DML



Données

- ✓ Insérer données
- ✓ Modifier données
- ✓ Supprimer données

DQL



Requêtes

- ✓ Sélectionner données
- ✓ Filtrer résultats
- ✓ Agréger informations



Le DDL travaille sur le **schéma** de la base de données, contrairement au DML et DQL qui manipulent les **données** elles-mêmes.

7.3 Création d'une base de données



CREATE DATABASE

Créer une nouvelle base

```
CREATE DATABASE universite;
```

Permet de créer une nouvelle base de données nommée **universite**.



USE

Sélectionner la base

```
USE universite;
```

Permet de sélectionner la base de données **universite** pour les opérations ultérieures.

Hiérarchie logique



Serveur SGBD

System de Gestion de Bases de Données



Base de Données

universite



Tables

ETUDIANT, MODULE, INSCRIPTION...

7.4 Création d'une table

<> Syntaxe générale

```
CREATE TABLE nom_table (  
    attribut1 type [contraintes],  
    attribut2 type [contraintes],  
    ...  
);
```

💡 Points clés

- **nom_table** : nom de la nouvelle table
- **attribut** : nom de la colonne
- **type** : type de données (INT, VARCHAR...)
- **contraintes** : PRIMARY KEY, NOT NULL...

🏠 Exemple simple

```
CREATE TABLE ETUDIANT (  
    id_etudiant INT,  
    nom VARCHAR(50),  
    prenom VARCHAR(50),  
    filiere VARCHAR(30)  
);
```

🔗 Schéma logique de la table

id_etudiant	nom	prenom	filiere
INT	VARCHAR(50)	VARCHAR(50)	VARCHAR(30)

7.5 Types de données (rappel important)

1 Types numériques

INT

Entier

Nombre entier (ex: 42, -7)

FLOAT

Nombre à virgule

Nombre réel (ex: 3.14, -0.5)

DECIMAL(p,s)

Nombre décimal

p=précision,

s=scale (ex:

DECIMAL(10,2))

Utilisé pour stocker des valeurs quantitatives

” Types caractères

CHAR(n)

Chaîne fixe

n caractères fixés (ex: CHAR(10))

VARCHAR(n)

Chaîne variable

Jusqu'à n caractères (ex: VARCHAR(50))

TEXT

Texte long

Chaîne de longueur variable illimitée

Utilisé pour stocker du texte

Types date

DATE

Date

AAAA-MM-JJ (ex: 2024-01-17)

TIME

Heure

HH:MM:SS (ex: 14:30:00)

DATETIME

Date et heure

AAAA-MM-JJ
HH:MM:SS

Utilisé pour stocker des informations temporelles

7.6 Contraintes d'intégrité en DDL



PRIMARY KEY

Clé primaire

Définition

- Identifie de façon unique chaque ligne
- Ne peut pas être NULL
- Une seule par table

```
CREATE TABLE ETUDIANT (  
  id_etudiant INT PRIMARY KEY,  
  nom VARCHAR(50),  
  prenom VARCHAR(50)  
);
```

Schéma

id_etudiant

nom
prenom



FOREIGN KEY

Clé étrangère

Définition

Réfère une clé primaire d'une autre table. Assure l'intégrité référentielle.

```
CREATE TABLE INSCRIPTION (  
  id_etudiant INT,  
  id_module INT,  
  FOREIGN KEY (id_etudiant)  
    REFERENCES ETUDIANT(id_etudiant)  
);
```

Intégrité référentielle

ETUDIANT

id_etudiant



INSCRIPTION

id_etudiant

7.6 Autres contraintes d'intégrité

NOT NULL

Empêche les valeurs nulles dans la colonne.

```
nom VARCHAR(50) NOT NULL
```

UNIQUE

Empêche les doublons dans la colonne.

```
email VARCHAR(50) UNIQUE
```

Schéma clé composite

INSCRIPTION

id_etudiant

id_module

note

= Clé primaire composée

CHECK

Impose une condition logique à la valeur.

```
age INT CHECK (age >= 18)
```

Composite

Clé primaire composée de plusieurs colonnes.

```
PRIMARY KEY (id_etudiant,  
id_module)
```

7.7 Création complète d'un schéma relationnel

MODULE

```
CREATE TABLE MODULE (  
  id_module INT PRIMARY KEY,  
  libelle VARCHAR(50),  
  credit INT CHECK (credit > 0)  
);
```

INSCRIPTION

```
CREATE TABLE INSCRIPTION (  
  id_etudiant INT,  
  id_module INT,  
  note FLOAT CHECK (note BETWEEN 0 AND 20),  
  PRIMARY KEY (id_etudiant, id_module),  
  FOREIGN KEY (id_etudiant)  
    REFERENCES ETUDIANT(id_etudiant),  
  FOREIGN KEY (id_module)  
    REFERENCES MODULE(id_module)  
);
```

Schéma global

ETUDIANT

id_etudiant
nom, prenom



INSCRIPTION

id_etudiant
id_module
note



MODULE

id_module
libelle, credit



Points clés du schéma

1. Table INSCRIPTION avec clé primaire composée
2. Contraintes CHECK sur crédit (>0) et note (0-20)
3. Intégrité référentielle vers ETUDIANT et MODULE

7.8 Modification d'une table (ALTER TABLE)



ADD

Ajouter un attribut

Ajoute une nouvelle colonne à la table existante

```
ALTER TABLE ETUDIANT  
ADD email VARCHAR(50);
```



La nouvelle colonne sera ajoutée à la fin de la table



MODIFY

Modifier un attribut

Change le type ou la longueur d'une colonne existante

```
ALTER TABLE ETUDIANT  
MODIFY email  
VARCHAR(100);
```



Attention : la modification peut affecter les données existantes



DROP

Supprimer un attribut

Supprime une colonne et toutes ses données

```
ALTER TABLE ETUDIANT  
DROP email;
```



Irréversible : toutes les données de la colonne seront perdues



Note importante : Les modifications avec ALTER TABLE affectent la structure de la table. Assurez-vous toujours de sauvegarder les données importantes avant d'apporter des modifications majeures.

7.9 Suppression des structures



DROP TABLE

Supprimer la table définitivement

```
DROP TABLE ETUDIANT;
```

Caractéristiques

- ✗ Supprime la table et toutes ses données
- ✗ Supprime la structure de la table
- ✗ Opération irréversible
- ✗ Plus lent que TRUNCATE



TRUNCATE TABLE

Vider la table, conserver la structure

```
TRUNCATE TABLE ETUDIANT;
```

Caractéristiques

- ✓ Supprime toutes les lignes de données
- ✓ Conserve la structure de la table
- ✓ Structure conservée pour réutilisation
- ✓ Opération plus rapide que DROP

↔ Tableau comparatif

DROP

Supprime la table

TRUNCATE

Vide la table

Irréversible

DROP

Rapide

TRUNCATE

7.10 Création de vues (VIEW) et Index



VIEW

Vue - Table virtuelle

Définition : Une vue est une table virtuelle basée sur une requête SELECT. Elle ne contient pas de données stockées, mais présente des données depuis une ou plusieurs tables.

```
CREATE VIEW vue_etudiants_info AS  
SELECT nom, prenom FROM ETUDIANT;
```

Schéma relationnel

ETUDIANT



VUE_ETUDIANTS_INFO



INDEX

Index - Accès rapide

Définition : Un index améliore la vitesse de recherche dans une table. Il fonctionne comme un index dans un livre, permettant un accès rapide aux données sans parcourir toute la table.

```
CREATE INDEX idx_nom  
ON ETUDIANT(nom);
```

Schéma d'accès

Table



Index



Accès rapide



Les index peuvent significativement améliorer les performances des requêtes SELECT

7.12 Bonnes pratiques en DDL



Noms clairs et explicites

- ✓ Utiliser des noms de tables et colonnes descriptifs
- ✓ Éviter les abréviations obscures
- ✓ Respecter une convention de nommage cohérente



Utiliser les contraintes

- ✓ Définir les clés primaires et étrangères
- ✓ Appliquer NOT NULL et UNIQUE quand nécessaire
- ✓ Utiliser CHECK pour valider les données



Normaliser avant créer

- ✓ Appliquer les formes normales (1NF, 2NF, 3NF)
- ✓ Éviter les redondances de données
- ✓ Assurer l'intégrité référentielle



Documenter le schéma

- ✓ Créer des commentaires pour chaque table et colonne
- ✓ Maintenir un dictionnaire de données
- ✓ Documenter les relations entre les tables



Éviter les modifications fréquentes

Une fois le schéma établi, minimisez les modifications structurelles. Chaque changement peut avoir des impacts sur les applications et requêtes existantes. Planifiez soigneusement la structure initiale pour éviter les ajustements constants.

7.13 Conclusion du Chapitre

Le DDL : Fondement des Bases de Données Relationnelles



Structure Fiable

- ✓ Définition précise du schéma
- ✓ Types de données adaptés
- ✓ Organisation logique



Intégrité des Données

- ✓ Contraintes PRIMARY KEY
- ✓ Intégrité référentielle
- ✓ Validation automatique



Manipulation Efficace

- ✓ Préparation au DML
- ✓ Optimisation DQL
- ✓ Performance garantie

Le DDL est la base indispensable pour construire des bases de données robustes, sécurisées et performantes.



Chapitre 8 : SQL – Langage de Manipulation de Données (DML)

Data Manipulation Language



INSERT



UPDATE



DELETE

SQL

Gestion du contenu des tables • Transactions et Contraintes

8.1 Introduction



Qu'est-ce que le DML ?

Data Manipulation Language

Le **Langage de Manipulation de Données** permet de **gérer le contenu des tables** d'une base de données.

Contrairement au DDL qui agit sur la structure, le DML agit sur les **données elles-mêmes**.



Capacités du DML



Insérer des données

Ajouter de nouvelles lignes dans les tables



Modifier des données existantes

Mettre à jour les valeurs des colonnes



Supprimer des données

Retirer des lignes des tables



Gérer les transactions

Valider ou annuler les opérations

8.2 Rôle du DML dans SQL

SQL

DDL



Structure

- ✓ Créer bases/tables
- ✓ Définir types
- ✓ Contraintes d'intégrité
- ✓ Modifier/Supprimer schéma

DML



Données

- ✓ Insérer données
- ✓ Modifier données
- ✓ Supprimer données
- ✓ Gérer les transactions

DQL



Requêtes

- ✓ Sélectionner données
- ✓ Filtrer résultats
- ✓ Agréger informations



Le DML est utilisé **quotidiennement** dans toutes les applications informatiques pour gérer le contenu des tables.

8.3 Insertion de données – INSERT



Insertion complète

Toutes les colonnes

```
INSERT INTO ETUDIANT  
VALUES (1, 'Ahmed', 'Ali', 'Informatique');
```



Insertion partielle

Colonnes spécifiques

```
INSERT INTO ETUDIANT  
(id_etudiant, nom)  
VALUES (2, 'Sara');
```



Insertion multiple

Plusieurs lignes

```
INSERT INTO ETUDIANT  
VALUES (3, 'Youssef', 'Ben',  
       'Math'),  
(4, 'Khadija', 'El', 'Info');
```



Note : L'ordre des valeurs doit correspondre à l'ordre des colonnes spécifiées.



Schéma logique

<> Requête INSERT

Commande d'insertion



Table ETUDIANT

Structure de données



Nouvelle ligne ajoutée

Données intégrées

8.4 Mise à jour des données – UPDATE



Syntaxe générale

```
UPDATE table  
SET attribut = valeur  
WHERE condition;
```



Exemple ciblé

```
UPDATE ETUDIANT  
SET filiere = 'Mathématiques'  
WHERE id_etudiant = 1;
```



⚠ Sans WHERE

Toutes les lignes de la table seront modifiées !



Schéma UPDATE



Table ETUDIANT

Données existantes



Filtrage (WHERE)

Sélection ciblée



Modification ciblée

Valeurs mises à jour

8.5 Suppression des données – DELETE



Syntaxe

```
DELETE FROM table  
WHERE condition;
```



Suppression ciblée

```
DELETE FROM ETUDIANT  
WHERE id_etudiant = 4;
```



Suppression totale

```
DELETE FROM ETUDIANT;
```

⚠ Toutes les lignes !



⚠ Avertissement

Utilisez toujours la clause WHERE pour supprimer uniquement les lignes souhaitées. Sans WHERE, toutes les données de la table seront perdues !



DELETE vs TRUNCATE

DELETE

- ✓ **Condition possible** : Peut filtrer avec WHERE
- ✓ **Annulable** : Peut être ROLLBACK
- ✓ **Lent** : Opère ligne par ligne

TRUNCATE

- ✗ **Pas de condition** : Vide toute la table
- ✗ **Non annulable** : Peut pas ROLLBACK
- ✓ **Rapide** : Supprime en bloc



💡 Bonne pratique

Utilisez **DELETE** pour des suppressions ciblées avec transactions, et **TRUNCATE** pour vider entièrement une table rapidement.

8.5 Comparaison DELETE vs TRUNCATE



DELETE

Suppression ciblée

- ✓ **Condition possible**
Peut filtrer avec WHERE pour supprimer des lignes spécifiques
- ✓ **Annulable (ROLLBACK)**
Peut être annulé dans une transaction si non commité
- ✗ **Lent**
Opère ligne par ligne avec journalisation

Idéal pour : Suppressions sélectives avec transactions et récupération possible



TRUNCATE

Suppression totale

- ✗ **Pas de condition**
Supprime toutes les lignes de la table en une seule opération
- ✗ **Non annulable**
Ne peut pas être annulé même dans une transaction
- ✓ **Rapide**
Supprime en bloc avec deallocation des pages

Idéal pour : Vider complètement une table rapidement sans récupération nécessaire

8.6 Transactions



Définition

Une **transaction** est une suite d'opérations DML exécutées comme **une unité logique**.



Propriétés ACID

A**Atomicité**

Tout ou rien

C**Cohérence**

État valide

I**Isolation**

Indépendance

D**Durabilité**

Persistance



Schéma d'une transaction

**Début**

```
START TRANSACTION;
```

**Opérations**

```
INSERT / UPDATE / DELETE
```

**Commit**

```
COMMIT;
```

**Rollback**

```
ROLLBACK;
```


8.7 COMMIT



Validation des modifications



COMMIT valide définitivement toutes les modifications effectuées dans une transaction. Les changements deviennent permanents et ne peuvent plus être annulés.

```
COMMIT;
```

✓ Permanent

Modifications sauvegardées

👁 Visible

Accessible aux autres sessions



Exemple d'utilisation

1 Démarrer transaction

```
START TRANSACTION;
```

2 Effectuer modifications

```
UPDATE ETUDIANT SET filiere='Info' WHERE id=1;  
INSERT INTO ETUDIANT VALUES (5, 'Fatima', 'Ali',  
'Chimie');
```

3 Valider définitivement

```
COMMIT;
```

8.8 ROLLBACK



Annulation des modifications



ROLLBACK annule toutes les modifications effectuées depuis le début de la transaction. La base de données revient à son état initial.

```
ROLLBACK;
```



Réversible

Retour à l'état initial



Annule tout

Modifications ignorées



Exemple d'utilisation

1 Démarrer transaction

```
START TRANSACTION;
```

2 Effectuer modifications

```
UPDATE ETUDIANT SET filiere='Info' WHERE id=1;  
DELETE FROM ETUDIANT WHERE id=4;
```

3 Annuler tout

```
ROLLBACK;
```

8.9 SAVEPOINT



Retour partiel



SAVEPOINT permet de créer un point de reprise dans une transaction. Il est possible de revenir à ce point en cas d'erreur ou d'annulation partielle.

```
SAVEPOINT sp1;  
UPDATE ETUDIANT SET filiere='Info';  
ROLLBACK TO sp1;
```



Flexible

Retour sélectif



Multiple

Plusieurs savepoints



Schéma SAVEPOINT



Transaction



SAVEPOINT sp1

```
SAVEPOINT sp1;
```



Opérations

```
INSERT / UPDATE
```



ROLLBACK TO

```
ROLLBACK TO sp1;
```



8.10 DML et contraintes d'intégrité



Contraintes et opérations DML



INSERT refuse les doublons

La contrainte **PRIMARY KEY** empêche l'insertion de valeurs en double

```
-- Erreur : clé primaire déjà existante
```



UPDATE respecte CHECK

La contrainte **CHECK** vérifie les conditions avant modification



DELETE respecte les clés étrangères

La contrainte **FOREIGN KEY** empêche la suppression d'une ligne référencée



Exemple d'erreur



Tentative d'insertion invalide

```
INSERT INTO INSCRIPTION VALUES (99, 1);
```



Erreur retournée

Intégrité référentielle violée : L'étudiant avec l'ID 99 n'existe pas dans la table **ETUDIANT**



Avantage : Les contraintes d'intégrité protègent automatiquement la base de données contre les incohérences

8.11 Exemple complet : scénario réel



Transfert bancaire

1. Démarrer transaction

```
START TRANSACTION;
```

2. Débiter le compte A

```
UPDATE COMPTE SET solde = solde - 500 WHERE id=1;
```

3. Créditer le compte B

```
UPDATE COMPTE SET solde = solde + 500 WHERE id=2;
```

4. Valider définitivement

```
COMMIT;
```

Atomicité garantie : Tout le transfert réussit ou rien du tout



Schéma bancaire



Compte A

id: 1

↑ -500 €

Débit



Compte B

id: 2

↓ +500 €

Crédit



Transfert



Résultat final

Transaction validée avec succès grâce à **COMMIT**

8.12 Bonnes pratiques DML



Utiliser WHERE

Toujours spécifier une clause **WHERE** pour limiter la portée des opérations UPDATE et DELETE

Sans WHERE, toutes les lignes sont affectées !



Tester avec SELECT

Exécuter d'abord un **SELECT** avec la même condition pour vérifier les données ciblées

Vérifier avant de modifier



Utiliser les transactions

Envelopper les modifications critiques dans des **transactions** pour garantir l'atomicité

COMMIT ou ROLLBACK



Éviter les suppressions massives

Préférer **DELETE** avec une condition précise plutôt que **TRUNCATE** pour des suppressions sélectives. Si suppression totale nécessaire, utiliser avec précaution et sauvegarde

❌ **DELETE** ciblé

❌ **TRUNCATE** total



Sauvegarder avant modifications

Toujours effectuer une **sauvegarde** de la base de données avant des opérations DML importantes ou des modifications structurelles

✅ **Sauvegarde de sécurité**

8.13 Conclusion du Chapitre



Le DML : Cœur opérationnel des bases de données



Gestion en temps réel

Le DML permet de manipuler les données instantanément, garantissant des opérations rapides et efficaces sur le contenu des tables



Cohérence assurée

Les transactions garantissent l'intégrité des données grâce aux propriétés ACID, évitant les incohérences et garantissant la fiabilité



Sécurité des opérations

Les mécanismes COMMIT, ROLLBACK et SAVEPOINT sécurisent les opérations critiques, permettant un contrôle total sur les modifications

+ INSERT

Ajouter des données

✎ UPDATE

Modifier des données

🗑 DELETE

Supprimer des données

↻ Transactions

Gérer les opérations



Chapitre 9 : SQL – Langage de Requête de Données (SELECT)

Data Query Language



Filtrer



Trier



Calculer

SQL

Extraction et analyse de données • Cœur du SQL

9.1 Introduction



Qu'est-ce que le DQL ?

Data Query Language

Le **Langage de Requête de Données** (DQL) est la partie du SQL qui permet d'interroger les bases de données.

☒ La commande principale du DQL est **SELECT**, considérée comme **le cœur du SQL**.

Rôle du DQL



Afficher les données

Extraire et visualiser les informations



Filtrer les résultats

Sélectionner les données pertinentes



Trier les informations

Organiser les résultats



Effectuer des calculs

Agrégation et statistiques

9.2 Rôle du SELECT dans SQL

 Afficher les données

Extraire et visualiser les informations

 Filtrer les résultats

Sélectionner les données pertinentes

 Trier les informations

Organiser selon des critères

 Effectuer des calculs

Agrégation et statistiques

 Regrouper les données

Agrégation par catégories

 Combiner plusieurs tables

Jointures et relations

Commande essentielle : `SELECT` est le cœur du SQL, utilisée dans 99% des requêtes pour extraire et analyser les données.

 Schéma général d'une requête

Requête `SELECT`



Analyse syntaxique



Optimisation



Exécution



Résultat

9.3 Syntaxe générale de SELECT

<> Structure d'une requête

- 1 `SELECT attributs`
- 2 `FROM table`
- 3 `WHERE condition`
- 4 `GROUP BY attributs`
- 5 `HAVING condition`
- 6 `ORDER BY attributs;`

Point important

- ✓ Les clauses sont **optionnelles**
- ⚠ L'ordre est **obligatoire**

Ordre d'exécution

- 1 FROM → Jointures
- 2 WHERE → Filtrage
- 3 GROUP BY → Regroupement
- 4 HAVING → Filtrage groupes
- 5 SELECT → Projection
- 6 ORDER BY → Tri

9.4 Sélection simple

■ Afficher toutes les colonnes

```
SELECT * FROM ETUDIANT;
```

 * = toutes les colonnes de la table

■ Afficher certaines colonnes

```
SELECT nom, prenom FROM ETUDIANT;
```

 Sélectionnez uniquement les colonnes nécessaires

🔍 Schéma de projection

📊 Table ETUDIANT

id

nom

prenom

filiere



🔍 Projection (colonnes)

Sélection des colonnes spécifiées



✅ Résultat

nom

prenom

9.5 Filtrage des données – WHERE

➡ Opérateurs de comparaison

=

Égal à

!=

Différent de

<

Inférieur à

>

Supérieur à

<=

Inférieur ou égal

>=

Supérieur ou égal

🔗 Opérateurs logiques

AND

ET logique

Toutes les conditions doivent être vraies

OR

OU logique

Au moins une condition doit être vraie

NOT

NON logique

Inverse la condition

<> Exemple complet

```
SELECT nom
FROM ETUDIANT
WHERE filiere = 'Informatique'
AND age >= 20;
```



Signification :

Affiche les noms des étudiants inscrits en Informatique ET âgés de 20 ans ou plus



Bonnes pratiques

- Utiliser des parenthèses pour clarifier l'ordre
- Tester les conditions progressivement



Attention

- Les chaînes de caractères sont entre apostrophes
- Les conditions sont évaluées de gauche à droite

9.6 Opérateurs avancés

↔ BETWEEN

```
WHERE note BETWEEN 10 AND 20;
```

Valeurs entre 10 et 20 inclus

⚡ IN

```
WHERE filiere IN ('Informatique',  
'Mathématiques');
```

Filières égales à l'une des
valeurs listées

T LIKE

```
WHERE nom LIKE 'A%';
```

Noms commençant par A

% = caractère générique (remplace 0 ou plusieurs caractères)

<> Schéma LIKE

→ Pattern

'A%' → Commence par A

→ Pattern

'%a' → Finit par a

→ Pattern

'%ab%' → Contient ab

i Caractère générique :

Remplace n'importe quel nombre de caractères

9.7 Tri des résultats – ORDER BY



Tri croissant

ORDER BY attribut ASC

```
SELECT * FROM ETUDIANT  
ORDER BY nom ASC;
```

↑ **ASC** = Ascendant (A → Z)



Par défaut :

Si ASC n'est pas précisé, le tri est croissant par défaut



Tri décroissant

ORDER BY attribut DESC

```
SELECT * FROM ETUDIANT  
ORDER BY nom DESC;
```

↓ **DESC** = Descendant (Z → A)



Exemples d'application :

- Notes les plus élevées en premier
- Dates les plus récentes en premier



Tri par ordre alphabétique : SELECT * FROM ETUDIANT ORDER BY nom;



Tri par valeur numérique : SELECT * FROM ETUDIANT ORDER BY age;

9.8 Fonctions SQL

Σ Fonctions d'agrégation



COUNT()
Nombre de lignes

```
COUNT(*)
```



SUM()
Somme des valeurs

```
SUM(note)
```



AVG()
Moyenne des valeurs

```
AVG(note)
```



MIN()
Minimum

```
MIN(note)
```



MAX()
Maximum

```
MAX(note)
```



Exemple :

```
SELECT AVG(note) FROM INSCRIPTION;
```



Schéma d'agrégation



Plusieurs lignes

10	15
12	18



Fonction



Calcule la moyenne



Valeur unique

13.75

Résultat de l'agrégation

9.9 GROUP BY

Définition

Regroupe les lignes selon un ou plusieurs attributs pour effectuer des calculs sur chaque groupe

Exemple complet

```
SELECT filiere, COUNT(*)
FROM ETUDIANT
GROUP BY filiere;
```

Signification :
Compte le nombre d'étudiants par filière

- Points clés :**
- Utilisé avec des fonctions d'agrégation (COUNT, SUM, AVG...)
 - Regroupe les lignes ayant les mêmes valeurs
 - Produit une ligne par groupe

Schéma GROUP BY

Données brutes

- Ahmed - Informatique
- Sara - Mathématiques
- Youssef - Informatique
- Khadija - Mathématiques



Groupes

Informatique	Ahmed, Youssef
Mathématiques	Sara, Khadija



Résultat par groupe

Filière	Count
Informatique	2
Mathématiques	2

9.10 HAVING

Définition

Permet de **filtrer les groupes** créés par GROUP BY, contrairement à WHERE qui filtre les lignes individuelles



WHERE

Filtre les **lignes** avant regroupement



HAVING

Filtre les **groupes** après regroupement

<> Exemple

```
SELECT filiere, COUNT(*)  
FROM ETUDIANT  
GROUP BY filiere  
HAVING COUNT(*) > 5;
```



Signification :

Affiche les filières ayant plus de 5 étudiants



Point clé

- ✓ **HAVING** fonctionne uniquement avec les fonctions d'agrégation
- ✓ **HAVING** est évalué après **GROUP BY**
- ✓ **WHERE** est évalué avant **GROUP BY**

9.11 Jointures (JOIN)

📌 Définition

Combine les données provenant de plusieurs tables selon des relations

↔ Jointure interne (INNER JOIN) Intersection des tables

```
SELECT E.nom, M.libelle
FROM ETUDIANT E
INNER JOIN INSCRIPTION I
    ON E.id_etudiant = I.id_etudiant
INNER JOIN MODULE M
    ON I.id_module = M.id_module;
```

🔗 Schéma INNER JOIN

ETUDIANT

INSCRIPTION

MODULE

↗ Jointure externe (LEFT JOIN) Tous les enregistrements de la table gauche

```
SELECT E.nom, I.note
FROM ETUDIANT E
LEFT JOIN INSCRIPTION I
    ON E.id_etudiant = I.id_etudiant;
```

🔗 Schéma LEFT JOIN

ETUDIANT
Table principale



INSCRIPTION
Table liée

📌 **Avantage :** Conserve tous les étudiants même sans inscription

9.12 Sous-requêtes

Définition

Une sous-requête est une requête imbriquée dans une autre, utilisée comme condition ou source de données

Exemple

```
SELECT nom
FROM ETUDIANT
WHERE id_etudiant IN (
    SELECT id_etudiant
    FROM INSCRIPTION
    WHERE note >= 10
);
```

 **Signification :**
Étudiants avec une note ≥ 10

Schéma de fonctionnement

↓ Sous-requête

```
SELECT id_etudiant FROM INSCRIPTION WHERE note >= 10
```

 Identifiants des étudiants réussis



Résultat intermédiaire

id_etudiant
1
3
5



✓ Requête principale

```
WHERE id_etudiant IN (1, 3, 5)
```

 Étudiants filtrés

9.13 DISTINCT et Alias

DISTINCT

Supprime les doublons dans le résultat

```
SELECT DISTINCT filiere  
FROM ETUDIANT;
```

 **Résultat :**
Affiche chaque filière une seule fois

Utilisation typique

- ✓ Compter les valeurs uniques
- ✓ Identifier les catégories
- ✓ Éliminer les répétitions

Alias (AS)

Renomme les colonnes dans le résultat

```
SELECT nom AS Nom_Etudiant  
FROM ETUDIANT;
```

 **Résultat :**
La colonne affichée s'appelle "Nom_Etudiant"

Avantages

- Lisibilité accrue
- Clarté du résultat

Cas d'usage

- Expressions calculées
- Fonctions SQL

9.15 Bonnes pratiques SELECT

❌ Éviter SELECT *

Sélectionnez uniquement les colonnes nécessaires

✗ Éviter :

```
SELECT * FROM ETUDIANT;
```

✓ Préférer :

```
SELECT nom, prenom FROM ETUDIANT;
```

📌 **Améliore** : Performance, Lisibilité, Maintenance

🔍 Indexer les colonnes filtrées

Créez des index sur les colonnes utilisées dans WHERE et JOIN

- ✓ Colonne de jointure (FOREIGN KEY)
- ✓ Colonne de filtrage (WHERE)
- ✓ Colonne de tri (ORDER BY)

➡ Utiliser JOIN plutôt que sous-requêtes lourdes

Les jointures sont généralement plus performantes

```
SELECT E.nom, M.libelle FROM ETUDIANT E INNER JOIN  
INSCRIPTION I ON E.id = I.id_etudiant INNER JOIN MODULE M  
ON I.id_module = M.id
```

➡ **Avantage** : Meilleure optimisation du plan d'exécution

🔧 Tester progressivement

Construisez et testez vos requêtes étape par étape

- 1 Commencez par la clause FROM
- 2 Ajoutez les jointures (JOIN)
- 3 Appliquez les filtres (WHERE)
- 4 Terminez avec le tri (ORDER BY)

9.16 Conclusion du Chapitre

SELECT : L'outil le plus puissant du SQL



Extraire

Récupérer les données pertinentes depuis les tables



Analyser

Effectuer des calculs et des regroupements



Décider

Prendre des décisions basées sur les données

Capacités du DQL



✓ Filtrage avec WHERE

✓ Regroupement GROUP BY

✓ Tri avec ORDER BY

✓ Jointures multiples

✓ Calculs d'agrégation

✓ Sous-requêtes



Chapitre 10 : Le SGBD MySQL

Système de Gestion de Bases de Données Relationnel



Open Source



Performant



Multi-utilisateurs

Architecture client/serveur • Moteurs de stockage • Transactions

10.1 Introduction



MySQL

SGBDR très populaire

MySQL est un **Système de Gestion de Bases de Données Relationnel** (SGBDR) open source.

Caractéristiques : Performant • Facile à utiliser • Open source



Domaines d'utilisation



Applications Web

PHP, Java, Python, Node.js



Systèmes d'Information

Gestion interne, ERP, CRM



Projets Académiques

Universités, Formations, Recherche



E-commerce

Boutiques en ligne, Marketplaces

10.2 Présentation générale de MySQL



Définition

MySQL est un **SGBD relationnel** qui combine performance et simplicité



Caractéristiques principales



Stockage en tables

Données structurées et relationnelles



Utilisation du SQL

Langage standard d'interrogation



Accès multi-utilisateurs

Connexions simultanées sécurisées



Cohérence et sécurité

Intégrité des données et protection



Domaines d'utilisation



Sites Web Dynamiques

PHP • Java • Python



E-commerce

Boutiques en ligne



Systèmes Universitaires

Gestion académique



Applications Client/Serveur

Architecture distribuée

Avantages clés



Open source • Gratuit • Communauté active • Documentation complète

10.3 Architecture de MySQL



Architecture Client/Serveur

MySQL fonctionne selon une architecture en 3 niveaux



10.3.2 Composants internes

Composants du serveur



Gestionnaire de connexions

Gère les connexions client



Analyseur SQL

Valide et analyse les requêtes



Optimiseur de requêtes

Améliore les performances



Moteur de stockage

Gère les données physiques



Gestionnaire de transactions

Assure l'atomicité

Schéma interne simplifié



Requête SQL

SELECT, INSERT, UPDATE, DELETE



Analyseur

Validation syntaxique



Optimiseur

Plan d'exécution optimal



Moteur de stockage

Accès et manipulation des données



Données

Tables, Index, Stockage physique

10.4 Moteurs de stockage MySQL

Schéma moteur de stockage



Table

Structure logique



Moteur

InnoDB / MyISAM / MEMORY



Fichiers disque

Stockage physique



InnoDB (par défaut)

Recommandé pour la majorité des applications

- ✓ Support des transactions
- ✓ Clés étrangères
- ✓ Verrouillage au niveau ligne
- ✓ Forte fiabilité



MyISAM

- 🚀 Rapide en lecture
- ❌ Pas de transactions
- ❌ Pas de clés étrangères



Autres moteurs

- ⚙️ MEMORY (en RAM)
- 📄 CSV (fichiers texte)
- 📁 ARCHIVE (historique)

Conseil



Utilisez InnoDB par défaut. MyISAM pour les applications en lecture seule ou nécessitant des performances optimales en lecture.

10.5 Installation et accès à MySQL



Accès via terminal

Interface en ligne de commande

```
mysql -u root -p
```



-u root

Spécifie l'utilisateur (root)



-p

Demande le mot de passe



Se connecter à MySQL

Accès direct au serveur MySQL



Outils graphiques

Interface visuelle conviviale



MySQL Workbench

Officiel

- ✓ Interface moderne
- ✓ Visualisation des schémas
- ✓ Gestion des utilisateurs



phpMyAdmin

Basé sur le web

- ✓ Accessible via navigateur
- ✓ Gestion des bases/tables
- ✓ Exécution de requêtes SQL

10.6 Gestion des bases de données



Créer

CREATE DATABASE

```
CREATE DATABASE  
université;
```



Nom unique

Doit être unique sur le serveur



Espace disque

Alloue l'espace nécessaire



Afficher

SHOW DATABASES

```
SHOW DATABASES;
```



Liste complète

Affiche toutes les bases



Vérification

Confirme la création



Supprimer

DROP DATABASE

```
DROP DATABASE  
université;
```



Irréversible

Supprime définitivement



Données perdues

Toutes les tables supprimées

10.7 Gestion des tables



Afficher les tables

SHOW TABLES

```
SHOW TABLES;
```



Liste des tables

Affiche toutes les tables de la base actuelle



Vérification rapide

Permet de confirmer les tables existantes

Note

Nécessite d'avoir sélectionné une base avec `USE`



Décrire une table

DESCRIBE

```
DESCRIBE ETUDIANT;
```



Structure complète

Colonnes, types, clés, contraintes



Informations détaillées

Nom, type, null, clé, default, extra



Alternative

Peut aussi utiliser `EXPLAIN` pour plus de détails

10.8 Gestion des utilisateurs et privilèges



Schéma de sécurité

Contrôle d'accès MySQL



Utilisateur

Identité et authentification



Privilèges

Droits et autorisations



Base / Tables

Ressources accessibles



Créer un utilisateur

CREATE USER

```
CREATE USER 'user1'@'localhost' IDENTIFIED BY 'password';
```



Attribuer les droits

GRANT

```
GRANT ALL PRIVILEGES ON universite.* TO 'user1'@'localhost';
```



Retirer les droits

REVOKE

```
REVOKE ALL PRIVILEGES ON universite.* FROM  
'user1'@'localhost';
```

10.9 Transactions dans MySQL



Schéma transactionnel

Flux d'une transaction MySQL



Début

START TRANSACTION



Opérations SQL

INSERT / UPDATE / DELETE



COMMIT

Valider



ROLLBACK

Annuler



MySQL avec moteur InnoDB

Les transactions nécessitent le moteur InnoDB pour garantir l'atomicité et la cohérence des données



Exemple complet

Transfert bancaire sécurisé

```
START TRANSACTION;  
UPDATE COMPTE SET solde = solde - 100 WHERE id=1;  
UPDATE COMPTE SET solde = solde + 100 WHERE id=2;  
COMMIT;
```



Atomicité



Cohérence



Isolation

10.10 Sauvegarde et restauration



Schéma sauvegarde

Processus de protection des données



Base de Données

Données actives MySQL



Sauvegarde

Fichier SQL généré



Restauration

Reconstitution des données



Sauvegarde

mysqldump

```
mysqldump universite > sauvegarde.sql
```



Structure



Données



Contraintes



Restauration

mysql

```
mysql universite < sauvegarde.sql
```



Important

La base doit exister avant la restauration. Les données existantes seront écrasées.

10.11 Performances et index



Création d'un index

CREATE INDEX

```
CREATE INDEX idx_nom ON ETUDIANT(nom);
```



Accélération



Performance



Bonnes pratiques

- ✓ Indexer les colonnes utilisées dans WHERE et JOIN
- ✓ Éviter les index sur des colonnes peu discriminantes
- ✓ Les index consomment de l'espace disque



Effet de l'index

Comparaison de performance



Sans index

Recherche lente

Balayage complet de la table

🕒 Temps d'exécution élevé



Avec index

Recherche rapide

Accès direct via l'index

🕒 Temps d'exécution réduit

10.12 Avantages et limites de MySQL



Avantages

Points forts de MySQL



Open source

Gratuit et modifiable, communauté active



Facile à apprendre

Documentation complète, nombreux tutoriels



Rapide

Performances optimales pour la plupart des cas



Large communauté

Support technique, forums, partage d'expériences



Limites

Points à considérer



Moins avancé qu'Oracle

Pour les très grands systèmes et entreprises



Fonctionnalités avancées limitées

Comparé aux SGBD commerciaux haut de gamme



Scalabilité

Complexité accrue avec les systèmes très volumineux



Idéal pour

Projets web, systèmes de taille moyenne, apprentissage

10.13 Exemple complet : mini système universitaire



Tables du système

Structure relationnelle



ETUDIANT

```
id_etudiant INT PRIMARY KEY
nom VARCHAR(50)
prenom VARCHAR(50)
filieres VARCHAR(30)
```



MODULE

```
id_module INT PRIMARY KEY
libelle VARCHAR(50)
credit INT
```



INSCRIPTION

```
id_etudiant INT FK
id_module INT FK
note FLOAT
```



Utilisation



Création

CREATE TABLE



Insertion

INSERT INTO



Requêtes

SELECT / JOIN



Transactions

START / COMMIT



Exemple de code

Création et utilisation

```
CREATE TABLE ETUDIANT (
  id_etudiant INT PRIMARY KEY,
  nom VARCHAR(50),
  prenom VARCHAR(50),
  filieres VARCHAR(30)
);
```

```
INSERT INTO ETUDIANT VALUES
(1, 'Ahmed', 'Ali', 'Info'),
(2, 'Sara', 'Ben', 'Math');
```

```
SELECT E.nom, M.libelle
FROM ETUDIANT E
JOIN INSCRIPTION I
ON E.id_etudiant = I.id_etudiant
JOIN MODULE M
ON I.id_module = M.id_module;
```

```
START TRANSACTION;
UPDATE ETUDIANT
SET filieres='Info'
WHERE id_etudiant=1;
COMMIT;
```

10.14 Conclusion du Chapitre

MySQL : SGBD relationnel puissant et accessible



Apprentissage

Idéal pour les débutants et les étudiants en informatique



Projets Web

Utilisé dans la majorité des applications web dynamiques



Systèmes de taille moyenne

Adapté aux systèmes d'information d'entreprise



Excellente transition

MySQL constitue une base solide pour évoluer vers des SGBD plus complexes comme Oracle, PostgreSQL ou SQL Server



Open Source



Performant



Fiable



Communauté active



Chapitre 11 : Le SGBD Oracle

Système de Gestion de Bases de Données Relationnel et Objet-Relationnel



Entreprise



Sécurité



Performance

Oracle

Architecture Instance/Base • Haute disponibilité • Transactions ACID

11.1 Introduction



Oracle Database

SGBDR & Objet-Relationnel

Système de Gestion de Bases de Données relationnel et objet-relationnel de niveau entreprise



Grandes entreprises



Banques et assurances



Administrations



Systèmes critiques



Caractéristiques clés



Robustesse

Fiabilité éprouvée pour les environnements de production



Sécurité avancée

Protection des données sensibles et conformité



Hautes performances

Traitement rapide de grandes quantités de données



Solution de niveau entreprise pour systèmes critiques

11.2 Présentation générale d'Oracle



Définition

SGBDR de niveau entreprise



Stockage structuré

Organisation optimisée des données



Accès simultané

Multi-utilisateurs concurrentiels



Gestion avancée

Transactions ACID complètes



Sécurité renforcée

Protection et haute disponibilité



Domaines d'utilisation

Applications critiques



Banques

Systèmes financiers et assurances



Télécommunications

Réseaux et opérateurs



Gouvernement

Systèmes administratifs



Industrie

Grandes bases de données



SGBDR robuste pour environnements critiques

11.3 Architecture d'Oracle



Vue générale

Architecture en deux éléments

L'architecture Oracle repose sur **deux composants principaux** qui collaborent pour assurer la gestion des données



Instance Oracle

Structures mémoire et processus



Base de données Oracle

Structures physiques de stockage



Schéma global Oracle



Utilisateur

Application / Client



Instance Oracle

Mémoire + Processus



Base de Données Oracle

Fichiers physiques

11.4 L'Instance Oracle



Définition

Cœur du système Oracle

L'instance Oracle est l'ensemble des **structures mémoire** et des **processus** permettant l'accès à la base



Structures mémoire

SGA et PGA



SGA

Mémoire partagée entre les utilisateurs



PGA

Mémoire privée pour chaque processus



Schéma mémoire



Instance Oracle



SGA

System Global Area



Buffer Cache



Shared Pool



Redo Log Buffer



PGA

Program Global Area

Mémoire privée pour chaque processus utilisateur

11.5 La base de données Oracle



Structures physiques



Datafiles

Fichiers de données



Redo Log Files

Journal des modifications



Control Files

Fichiers de contrôle



Stockage physique des données sur le disque



Structures logiques



Tablespaces

Unités logiques de stockage



Segments

Ensembles d'extents pour les objets



Extents

Blocs contigus de stockage



Blocs

Unités minimales de stockage



Schéma logique

Tablespace



Segment



Extent



Bloc

11.6 Tablespaces



Définition

Unité logique de stockage

Un tablespace est une **unité logique de stockage** qui regroupe un ou plusieurs fichiers physiques pour stocker les données des tables et index



Rôle : Les tablespaces organisent l'espace disque et facilitent la gestion de la base de données



Tablespace Système

Données système Oracle



Tablespace Utilisateur

Tables et index utilisateur

Tablespace Temporaire

Opérations temporaires



Tablespace Undo

Gestion des transactions



Exemple de création

Syntaxe Oracle

```
CREATE TABLESPACE ts_data  
DATAFILE 'data01.dbf'  
SIZE 100M;
```

1

Nom du tablespace

Identifier unique : ts_data

2

Fichier physique

Fichier de données : data01.dbf

3

Taille initiale

Espace alloué : 100 mégaoctets

11.7 Gestion des utilisateurs Oracle



Création d'un utilisateur

CREATE USER

```
CREATE USER admin  
IDENTIFIED BY admin123;
```

 Crée un nouveau compte utilisateur avec mot de passe



Attribution des privilèges

GRANT

```
GRANT CONNECT, RESOURCE  
TO admin;
```

CONNECT

Connexion à la base

RESOURCE

Création d'objets



Schéma de sécurité



Utilisateur

Compte Oracle



Rôles

Ensemble de privilèges



Privilèges

Droits d'accès



Avantages

- Séparation claire des rôles et des droits
- Gestion simplifiée des permissions
- Sécurité renforcée par le principe du moindre privilège

11.8 Objets de la base Oracle



Séquences

Génération automatique

Génèrent des valeurs automatiques et uniques

```
CREATE SEQUENCE seq_etudiant
START WITH 1
INCREMENT BY 1;
```

Avantage : Remplace les triggers pour l'auto-incrémentation



Synonymes

Alias pour objets

Alias pour simplifier l'accès aux objets

```
CREATE SYNONYM etu
FOR ETUDIANT;
```

Avantage : Simplifie les requêtes et masque la structure



Index

Optimisation

Améliorent la vitesse de recherche

```
CREATE INDEX idx_nom
ON ETUDIANT(nom);
```

Avantage : Accélération des requêtes SELECT



Utilisation

seq_etudiant.NEXTVAL pour obtenir la valeur suivante



Simplification

SELECT * FROM etu au lieu de SELECT * FROM ETUDIANT



Performance

Index sur colonnes fréquemment utilisées dans WHERE

11.9 Transactions dans Oracle



Propriétés ACID

Garantie de fiabilité

A

Atomicité

C

Cohérence

I

Isolation

D

Durabilité



Exemple de transaction

Transfert bancaire sécurisé

```
UPDATE COMPTE
SET solde = solde - 100
WHERE id = 1;
COMMIT;
```



Schéma transactionnel



Transaction

Opérations SQL en cours



Redo Log

Journal des modifications



Commit

Validation définitive



Points clés : Oracle enregistre toutes les modifications dans le Redo Log avant validation, garantissant la récupération en cas de panne

11.10 Sauvegarde et récupération



Sauvegarde logique (export)

expdp - Data Pump Export

```
expdp system/password  
schemas=admin  
dumpfile=backup.dmp
```



Exportation de schéma

Sauvegarde de toutes les tables et objets du schéma admin



Fichier de sauvegarde

Format binaire optimisé : backup.dmp



Récupération (import)

impdp - Data Pump Import

```
impdp system/password  
schemas=admin  
dumpfile=backup.dmp
```



Restauration de schéma

Importation de toutes les tables et objets



Restauration complète

Recréation de la structure et des données



Expdp

- Sauvegarde rapide et efficace
- Supporte les très grandes bases de données
- Compression intégrée



Impdp

- Restauration précise
- Préserve les métadonnées
- Supporte le parallélisme

11.11 Performances et optimisation



Index
Accélération des recherches



Recherche rapide
Accès direct via index B-tree

CREATE INDEX

Création d'index
Sur colonnes fréquemment utilisées

maintain

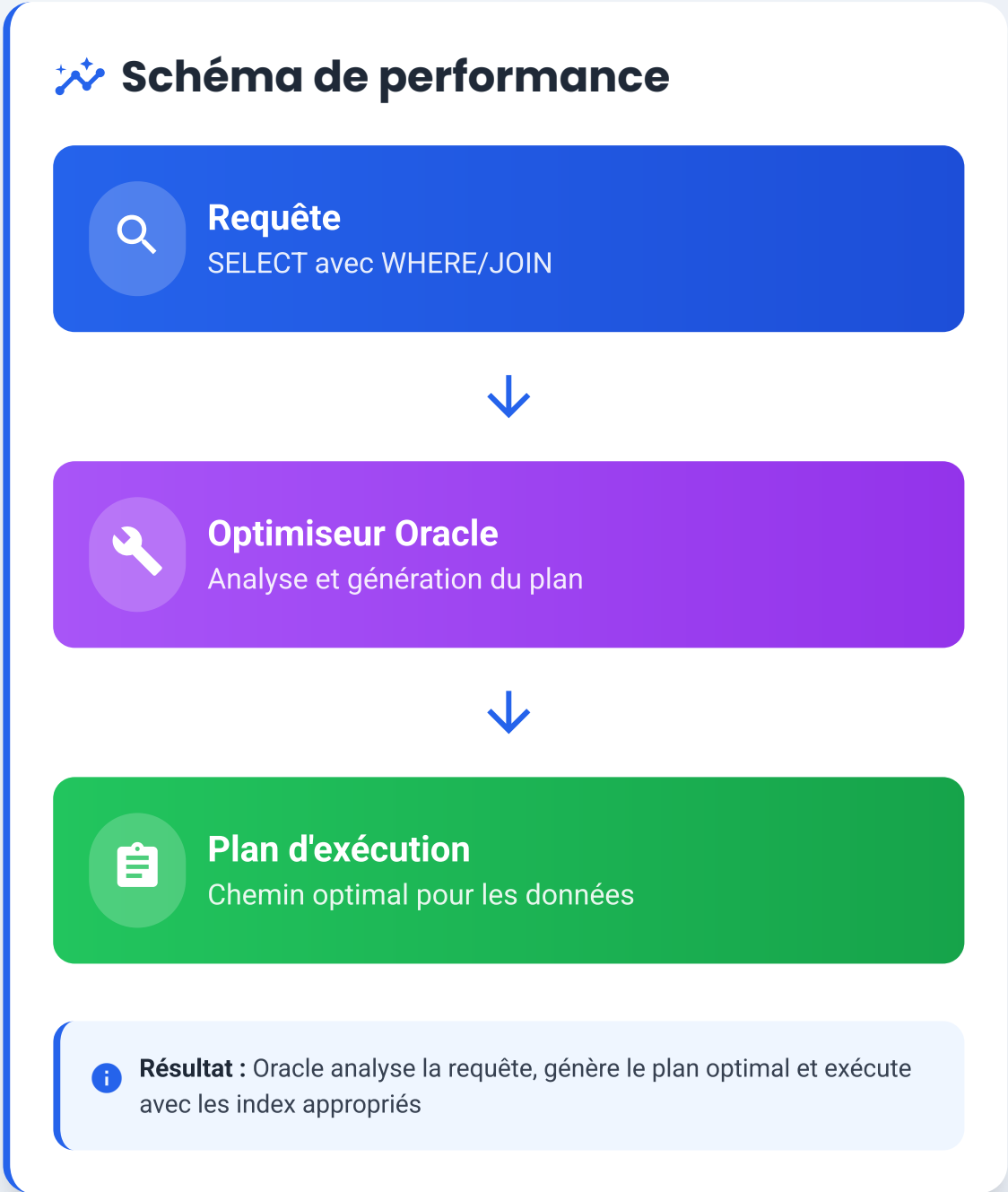
Maintenance
Statistiques à jour pour l'optimiseur



Plans d'exécution
Analyse et optimisation automatique des requêtes



Partitionnement
Division des grandes tables en partitions



11.12 Avantages et limites d'Oracle



Avantages



Très grande fiabilité

Système éprouvé avec plus de 40 années d'expérience en entreprise. Haute disponibilité garantie pour les systèmes critiques.



Sécurité avancée

Sécurité au niveau entreprise avec chiffrement, audit complet, authentification multifacteur et conformité aux normes.



Support des grandes bases

Gère des pétaoctets de données avec des milliers d'utilisateurs simultanés sans perte de performance.



Limites



Coût élevé

Licences onéreuses et infrastructure requise. Maintenance et support techniques spécialisés.



Complexité d'administration

Courbe d'apprentissage élevée. Nécessite des administrateurs Oracle certifiés pour la gestion optimale.



Recommandation : Oracle est idéal pour les entreprises ayant des budgets conséquents et des besoins de haute fiabilité. MySQL ou PostgreSQL sont préférables pour les projets avec des contraintes budgétaires.

11.13 Exemple complet : environnement bancaire



Comptes

Gestion des comptes clients



Compte courant

Transactions quotidiennes, virements, retraits

savings

Compte épargne

Intérêts cumulés, plafonds de retrait



Transactions

Opérations bancaires sécurisées

arrow_upward

Virement

Transfert interbancaire instantané

payment

Paie ment

Cartes bancaires, prélèvements



Sécurité

- ✓ Chiffrement des données
- ✓ Authentification multifacteur
- ✓ Audit et journalisation



Haute disponibilité

- ✓ Redondance des serveurs
- ✓ Basculement automatique
- ✓ Sauvegarde en temps réel



Architecture système bancaire



Application
Bancaire



Oracle
RAC



Datafiles
+ Redo Logs



Oracle RAC (Real Application Clusters) : Assure la continuité de service même en cas de panne d'un serveur, garantissant que les services bancaires restent accessibles 24/7

11.14 Conclusion du Chapitre

SGBD Oracle de niveau entreprise



Haute disponibilité

Disponibilité 24/7 garantie par l'architecture RAC (Real Application Clusters)



Sécurité renforcée

Chiffrement, audit complet et authentification multifacteur



Performances

Optimiseur intelligent et gestion avancée des transactions



Idéal pour les systèmes critiques



Banques et assurances



Administrations



Grandes entreprises



Oracle : La solution pour systèmes de niveau entreprise

Fiabilité éprouvée • Sécurité intégrée • Performance optimale • Support des bases massives



Oracle Database

CHAPITRE 12

PL/SQL

Programmation en Base de Données

Procedural Language / SQL

12.1 Introduction au PL/SQL

Procedural Language / SQL

Langage Procédural Oracle

Permet d'ajouter une logique de programmation au SQL classique



Automatiser



Règles métier

PL
SQL

PL/SQL est utilisé pour :

1

Automatiser les traitements

Exécution automatique de tâches complexes

2

Implémenter des règles métier

Validation et contrôle des données

3

Améliorer les performances

Traitements optimisés côté serveur

4

Sécuriser les opérations

Contrôle d'accès et validation

#1

12.2 Différence entre SQL et PL/SQL

Comparaison

PL/SQL étend SQL en ajoutant des fonctionnalités procédurales pour une programmation avancée

SQL

- Déclaratif
- Une instruction à la fois
- Pas de conditions
- Pas de variables

PL/SQL

- Procédural
- Blocs d'instructions
- IF, LOOP
- Variables supportées

<> SQL

 PL/SQL



PL/SQL combine la puissance de SQL avec des structures de programmation avancées

12.3 Structure d'un bloc PL/SQL

Syntaxe Générale

1

DECLARE

-- Déclaration des variables (optionnel)

2

BEGIN

-- Instructions (obligatoire)

3

EXCEPTION

-- Gestion des erreurs (optionnel)

4

END;

-- Fin du bloc

Schéma du Bloc PL/SQL



DECLARE

Optionnel



BEGIN

Obligatoire



EXCEPTION

Optionnel



END

Fin du bloc

12.4 Variables et Types en PL/SQL

Déclaration de Variables

```
<> v_nom ETUDIANT.nom%TYPE;
```

Déclencheur : héritage du type de colonne

```
<> v_age NUMBER;
```

Déclaration explicite du type

i Les variables peuvent hériter du type d'une colonne existante ou être explicitement typées

Types de Données Principaux

NUMBER

Nombres entiers et décimaux

Tt VARCHAR2

Chaînes de caractères

📅 DATE

Dates et heures

🔘 BOOLEAN

Valeurs logiques (VRAI/FAUX)

12.5 Instructions Conditionnelles

IF – THEN – ELSE

```
IF note >= 10 THEN  
    resultat := 'Admis';  
ELSE  
    resultat := 'Ajourné';  
END IF;
```

💡 La structure conditionnelle permet d'exécuter différentes actions selon la valeur de la condition

Schéma Conditionnel

✓ **Condition**
Test logique



👍 **Vrai**
Action principale

👎 **Faux**
Action alternative

12.6 Boucles (LOOPS)

LOOP

Boucle simple

```
LOOP  
  EXIT WHEN i > 5;  
  i := i + 1;  
END LOOP;
```

WHILE

Boucle conditionnelle

```
WHILE i <= 5 LOOP  
  i := i + 1;  
END LOOP;
```

FOR

Boucle itérative

```
FOR i IN 1..5 LOOP  
  DBMS_OUTPUT.PUT_LINE(i);  
END LOOP;
```

Choisir la bonne boucle

LOOP Quand le nombre d'itérations est inconnu

WHILE Tant que la condition est vraie

FOR Pour un nombre d'itérations défini

12.7 Curseurs

Définition

 Un curseur permet de parcourir les lignes retournées par une requête SELECT

✦ Curseur Implicite

```
SELECT * INTO v_nom FROM ETUDIANT WHERE  
id=1;
```

⚙ Curseur Explicite

```
CURSOR c_etudiant IS SELECT nom FROM  
ETUDIANT;
```

Schéma du Curseur

 **SELECT**
Requête SQL



 **Résultat**
Ensemble de données



 **Parcours**
Ligne par ligne

12.8 Procédures Stockées

Définition

Une procédure est un programme stocké dans la base de données

✓ Avantages

- Réutilisabilité du code
- Sécurité renforcée
- Amélioration des performances

Exemple de Procédure

```
CREATE PROCEDURE afficher_etudiants IS  
BEGIN  
    DBMS_OUTPUT.PUT_LINE('Liste des étudiants');  
END;
```

<> Explication

CREATE PROCEDURE Création de la procédure

afficher_etudiants Nom de la procédure

DBMS_OUTPUT.PUT_LINE Affichage en sortie

12.9 Fonctions

Définition

Σ Une fonction retourne une valeur

★ Caractéristiques

- Retourne une seule valeur
- Utilisable dans les requêtes SQL
- Réutilisable et performante

Exemple de Fonction

```
CREATE FUNCTION calcul_moyenne RETURN NUMBER IS  
BEGIN  
    RETURN 12;  
END;
```

<> Explication

CREATE FUNCTION Création de la fonction

calcul_moyenne Nom de la fonction

RETURN NUMBER Type de retour (nombre)

RETURN 12 Valeur retournée

12.10 Triggers (Déclencheurs)

Définition

- ⚡ Un trigger est exécuté automatiquement lors d'un événement sur la base de données

🔔 Caractéristiques

- ▶ Exécution automatique
- ▶ Déclenchement par événement
- ▶ Validation des données

Exemple de Trigger

```
CREATE TRIGGER verific_note
BEFORE INSERT ON INSCRIPTION
FOR EACH ROW
BEGIN
    IF :NEW.note < 0 OR :NEW.note > 20 THEN
        RAISE_APPLICATION_ERROR(-20001, 'Note invalide');
    END IF;
END;
```

📅 **BEFORE INSERT**
Avant l'insertion

≡ **FOR EACH ROW**
Pour chaque ligne

12.11 Gestion des Exceptions

Principe

❗ Gérer les erreurs et exceptions dans le bloc PL/SQL

⚠ Exceptions Courantes

NO_DATA_FOUND

Aucune donnée trouvée

TOO_MANY_ROWS

Trop de lignes retournées

VALUE_ERROR

Erreur de conversion

Exemple Complet

```
BEGIN
    SELECT nom INTO v_nom FROM ETUDIANT WHERE id=99;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Étudiant inexistant');
END;
```

▶ BEGIN

Début du bloc

🔍 SELECT

Recherche étudiant

👁 WHEN

Capture erreur



Le bloc **EXCEPTION** permet de gérer les erreurs de manière gracieuse et de fournir des messages explicites à l'utilisateur.

12.12 Avantages du PL/SQL

Logique Métier

Intégration des règles métier directement dans la base de données pour une gestion centralisée et cohérente



Meilleures Performances

Exécution des traitements directement côté serveur réduisant les échanges réseau et améliorant la rapidité



Sécurité Renforcée

Contrôle d'accès granulaire et protection des données avec triggers et procédures stockées sécurisées



Modularité

Code structuré en procédures et fonctions réutilisables facilitant la maintenance et l'évolution des applications



12.13 Limites du PL/SQL



Spécifique à Oracle

Non portable vers d'autres systèmes de gestion de bases de données (SGBD) comme PostgreSQL, MySQL ou SQL Server

Vendor Lock-in



Complexité d'Apprentissage

Courbe d'apprentissage significative pour maîtriser la syntaxe, les concepts procéduraux et les mécanismes avancés

Technique Spécialisée



Dépendance au SGBD

Le code PL/SQL dépend entièrement de l'infrastructure Oracle, nécessitant des compétences et licences spécifiques

Architecture Propriétaire

12.14 Exemple Complet : Gestion des Notes



Application Pratique du PL/SQL

Cas d'utilisation réaliste dans un système éducatif



Contrôle des Valeurs

Validation

Note comprise entre 0 et 20

Trigger

Vérification BEFORE INSERT

```
IF note < 0 OR note > 20
```



Calcul Automatique

Moyenne

Calcul des notes moyennes

Résultat

Admis / Ajourné

```
IF moyenne >= 10 THEN 'Admis'
```



Messages d'Erreur

Gestion

Capture des exceptions

Feedback

Messages explicites

```
RAISE_APPLICATION_ERROR
```

PL/SQL : Transformer la Base de Données

PL/SQL permet de transformer une base de données en un **système intelligent**, capable d'automatiser, contrôler et sécuriser les données.



Automatiser

Exécution automatique des traitements



Contrôler

Validation et gestion des données



Sécuriser

Protection contre les erreurs et accès



Chapitre 13 : Sécurité et Administration des Bases de Données

CONFIDENTIALITÉ · INTÉGRITÉ · DISPONIBILITÉ



Confidentialité

Accès autorisé



Intégrité

Données intactes



Disponibilité

Accès constant

Introduction et Objectifs de Sécurité

Pourquoi la Sécurité?

⚠ Données Sensibles

- 👤 Données personnelles
- 🏦 Données financières
- 📅 Données administratives

🛡 Mécanismes de sécurité

🛡 Administration rigoureuse

Triade CIA



Confidentialité

Accès autorisé
uniquement aux
personnes
habilitées



Intégrité

Données ni
altérées ni
corrompues



Disponibilité

Données
accessibles en
cas de panne



Schéma des Objectifs

Confidentialité



Intégrité



Disponibilité

Authentification des Utilisateurs

Exemple MySQL

Définition



Vérification de l'identité avant l'accès

Moyens d'Authentification



Nom d'utilisateur / Mot de passe
Méthode classique



Authentification par Rôle
Privilèges groupés



Authentification Externe
LDAP, OS

```
CREATE USER 'user1'@'localhost'  
IDENTIFIED BY 'password';
```

Processus d'Authentification



Utilisateur

Demande d'accès



Identité Vérifiée

Validation réussie



Accès Autorisé

Connexion établie

Autorisation et Contrôle d'Accès

Définition

 Détermine ce qu'un utilisateur a le droit de faire après authentification

Types de Privilèges

Privilèges Système

CREATE

DROP

ALTER

Privilèges Objets

SELECT

INSERT

UPDATE

DELETE

Exemple : GRANT

```
GRANT SELECT, INSERT  
ON ETUDIANT  
TO user1;
```

Schéma des Droits

 **Utilisateur**
user1



 **Privilèges**
SELECT,
INSERT



 **Objets**
ETUDIANT

Rôles

Définition



Ensemble de privilèges
attribués à un utilisateur

Avantages



Simplification de la gestion
des droits



Meilleure organisation



Gestion centralisée des
permissions

Exemple Complet

```
CREATE ROLE enseignant;
```

```
GRANT SELECT ON ETUDIANT TO enseignant;
```

```
GRANT SELECT ON MODULE TO enseignant;
```

```
GRANT enseignant TO user1;
```

Structure des Rôles



Création du Rôle



Attribution des Privilèges



Attribution aux Utilisateurs

Sécurité des Données

Chiffrement

Rend les données illisibles sans clé de déchiffrement



Stockage

Chiffrement au repos

Transport

Chiffrement en transit

Vues Sécurisées

👁 Limite l'accès à certaines colonnes

Exemple : Création de Vue

```
CREATE VIEW vue_notes AS  
SELECT nom, note  
FROM ETUDIANT  
JOIN INSCRIPTION;
```

Avantages

- ✓ Masquer colonnes sensibles
- ✓ Simplifier requêtes complexes
- ✓ Contrôle granulaire des accès

Transactions et Contrôle de Concurrency

Problèmes de Concurrency

❗ Lecture Sale

Lecture de données non validées

🔄 Mise à jour Perdue

Écrasement d'une modification

🚫 Lecture Non Répétable

Données modifiées entre lectures

Verrouillage (Locking)

👁 Verrou de Lecture

Permet la lecture simultanée

✍ Verrou d'Écriture

Accès exclusif modification

Schéma de Verrouillage

👤 Utilisateur A

🔒 Verrou Actif

Accède aux données

👤 Utilisateur B

⌛ En Attente

Patiente libération

🚫 Évite les conflits d'accès simultané

Sauvegarde des Bases de Données

Importance

 Restaurer après panne

 Récupérer après erreur humaine

 Protéger contre les attaques

Types de Sauvegarde



Complète

Toutes les données



Différentielle

Modifications depuis
dernière complète



Incrémentielle

Depuis la dernière
sauvegarde

Processus de Sauvegarde



Base de Données

Données
actives



Sauvegarde

Processus de
copie



Support Externe

Disque
dur,
Cloud

Restauration des Données

Définition

Remettre la base dans un état valide

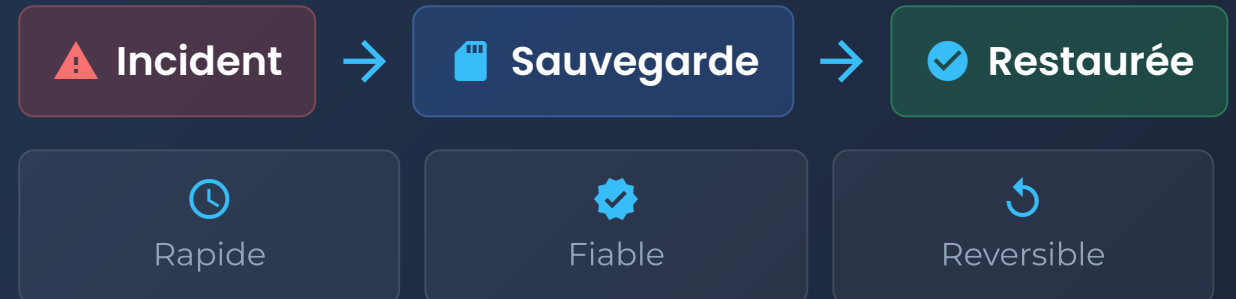
Importance

- * Récupération après panne critique
- 🕒 Retour à un état antérieur
- 🛡️ Intégrité des données garanties

Exemple MySQL

```
mysql universite < sauvegarde.sql
```

Processus de Restauration



Journalisation et Reprise après Incident

Journalisation (Logs)

Enregistre toutes les opérations de la base de données

Fonctions Principales

-  **Traçabilité**
Historique complet des opérations
-  **Récupération**
Restauration en cas de panne
-  **Audit**
Analyse des activités

Schéma du Processus



Avantages de la Journalisation

-  Récupération rapide
-  Historique complet
-  Sécurité accrue
-  Conformité légale

Surveillance et Audit

Audit



Permet de tracer les actions des utilisateurs



Tracer les actions

Historique des activités



Détecter les anomalies

Comportements suspects



Assurer la conformité

Régulations légales

Monitoring



Surveillance continue du système



Surveillance des performances

Temps de réponse, débit



Détection des pannes

Alertes en temps réel



Alertes automatiques

Notifications proactive

Rôle de l'Administrateur de Base de Données (DBA)

Définition

Le DBA est le gardien de la sécurité et de la performance de la base de données

Responsabilités Clés

-  Gestion des utilisateurs et des droits
-  Sécurité et protection des données
-  Sauvegarde et restauration
-  Optimisation des performances
-  Disponibilité du système

Schéma du Rôle du DBA



DBA



Sécurité

Protection des données



Sauvegarde

Protection continue



Performance

Optimisation



Gestion Utilisateurs



Disponibilité

Bonnes Pratiques de Sécurité

Objectifs



Garantir une sécurité continue et proactive



Approche Défensive

Prévenir plutôt que guérir



Mots de Passe Forts

Complexité, longueur, rotation



Privilèges Minimaux

Accès strictement nécessaire



Sauvegardes Régulières

Automatisées et testées



Audits Fréquents

Revue et tests de sécurité



Mises à Jour du SGBD

Patches et correctifs de sécurité

Exemple Réel : Système Bancaire

Contexte



Gestion des comptes
bancaires

Transactions sensibles
nécessitant une sécurité
maximale

Enjeux Critiques

-  Confidentialité absolue
-  Intégrité des montants
-  Disponibilité 24/7

Mise en Œuvre de la Sécurité



Accès Restreint aux Comptes

Authentification forte et rôles limités (titulaire, agent, administrateur)



Transactions Sécurisées

Verrouillage atomique, validation ACID, journalisation complète



Sauvegardes Journalières

Sauvegardes complètes nocturnes et incrémentielles horaires



Audit Permanent

Traçabilité de toutes les transactions et alertes automatiques

Conclusion du Chapitre

Points Clés



Fiabilité

Données fiables et cohérentes



Protection

Sécurité contre les menaces



Continuité

Disponibilité permanente

Message Final

Sans une **bonne administration**, même une base bien conçue peut devenir **vulnérable**.

! Rappel

La sécurité et l'administration ne sont pas des options, mais des **nécessités absolues** pour garantir l'intégrité et la pérennité de vos bases de données.

 **La sécurité est un processus continu**

Chapitre 14 : Révisions, Études de Cas et Projets

Consolidation et Mise en Pratique

Après l'étude complète des bases de données, des SGBD, de la modélisation, du SQL, de MySQL, Oracle, PL/SQL, de la sécurité et administration, il est essentiel de consolider les connaissances à travers :

 Révisions synthétiques

 Études de cas réelles

 Projets complets



Concepts



Modélisation



SQL



SGBD

Étapes du Projet

 Analyse

 Modélisation

 Implémentation

 Tests



Du Théorique au Pratique

Mise en œuvre de tous les concepts du cours

Introduction

Parcours du Cours

- 1 Bases de données
- 2 SGBD
- 3 Modélisation
- 4 SQL
- 5 MySQL / Oracle
- 6 Sécurité et Administration

Consolidation des Connaissances

✓ Trois Piliers

 **Révisions Synthétiques**
Consolidation des concepts

 **Études de Cas Réelles**
Mise en pratique concrète

 **Projets Complets**
Conception de A à Z

 Du savoir aux compétences
professionnelles

Révision Générale du Cours

Concepts Fondamentaux



Base de Données

Ensemble structuré de données permanentes



SGBD

Système de Gestion de Bases de Données



Indépendance des Données

Séparation programme / structure

Modélisation



Modèle Entité-Association

Entités

Attributs

Associations

Cardinalités



Héritage

Spécialisation / Généralisation

→ Du conceptuel au relationnel

Révision Continue – Modèle Relationnel et SQL

Modèle Relationnel



Relations

Tables avec colonnes et lignes



Clés

Primaire, Étrangère, Candidate



Intégrité Référentielle

Clé étrangère valide



Normalisation

1FN, 2FN, 3FN

Langage SQL



DDL – Définition

CREATE

ALTER

DROP



DML – Manipulation

INSERT

UPDATE

DELETE



DQL – Requête

SELECT

avec WHERE, JOIN, GROUP BY



Modèle EA → Relationnel → Normalisation → SQL

Étude de Cas 1 : Système Universitaire

Analyse des Besoins



Étudiants

Gestion des inscrits



Enseignants

Intervenants pédagogiques



Modules

Cours et formations



Notes & Inscriptions

Évaluation et suivi

Modélisation Conceptuelle EA

ETUDIANT

id, nom, filière



INSCRIPTION

MODULE

id, libelle, crédit



Attribut d'association : **note**

Associations Principales



INSCRIPTION

Étudiant ↔ Module



ENSEIGNE

Enseignant ↔ Module

Etude de Cas 1 : Modèle Relationnel et Implémentation SQL

Modèle Relationnel



ETUDIANT

(#id, nom, filiere)



MODULE

(#id, libelle)



INSCRIPTION

(#id_etudiant, #id_module, note)

Implémentation SQL

```
CREATE TABLE ETUDIANT (  
  id INT PRIMARY KEY,  
  nom VARCHAR(50) NOT NULL,  
  filiere VARCHAR(30)  
);  
CREATE TABLE MODULE (  
  id INT PRIMARY KEY,  
  libelle VARCHAR(50) NOT NULL,  
  credit INT  
);  
CREATE TABLE INSCRIPTION (  
  id_etudiant INT,  
  id_module INT,  
  note FLOAT,  
  PRIMARY KEY (id_etudiant, id_module),  
  FOREIGN KEY (id_etudiant)
```

 Clés Primaires

 Clés Étrangères

 Intégrité

Étude de Cas 2 : Système Bancaire

Gestion



Clients

Identité et coordonnées



Comptes

Soldes et numéros



Transactions

Mouvements financiers



Agences

Géographie du réseau

Contraintes Critiques



Sécurité Élevée

Accès restreint et protégé



Transactions Fiables

Atomicité et cohérence ACID



Historique Conservé

Traçabilité complète

Schéma Bancaire

CLIENT



COMPTE



TRANSACTION

Projet Académique : Conception Complète d'une Base

Objectif du Projet

Réaliser une base de données complète depuis :

- l'analyse
- la modélisation
- l'implémentation
- les tests

Étapes du Projet

1

Analyse des besoins

Collecte des exigences

2

Modélisation EA

Entité-Association

3

Modèle Relationnel

Schéma logique

4

Normalisation

1FN, 2FN, 3FN

5

Création SQL

DDL, DML

6

Insertion & Requêtes

SELECT, JOIN

7

Sécurité & Sauvegarde

Protection des données

Exemples de Sujets de Projets

Domaines d'Application

Projet Académique

Concevoir et implémenter une base de données complète :

- Analyse des besoins
- Modélisation EA
- Implémentation SQL
- Requêtes et sécurité



Bibliothèque

- Livres
- Emprunts
- Lecteurs



Hôpital

- Patients
- Médecins
- Rendez-vous



École

- Élèves
- Classes
- Notes



Magasin

- Produits
- Clients
- Ventes



Hôtel

- Chambres
- Réservations
- Clients

Exemples de Questions d'Examen

1

Indépendance des Données

Expliquer l'indépendance des données et son importance dans la conception des bases de données

2

DDL vs DML

Donner les différences entre le Langage de Définition de Données (DDL) et le Langage de Manipulation de Données (DML)

3

Normalisation

Normaliser la relation suivante jusqu'à la 3ème forme normale : COMMANDE(id, date, id_client, nom_client, articles)

4

Requête SQL avec JOIN

Écrire une requête SQL pour afficher le nom des étudiants avec le libellé des modules auxquels ils sont inscrits

5

Rôle du DBA

Expliquer le rôle de l'administrateur de base de données (DBA) et ses responsabilités principales

Méthodologie de Révision



Comprendre avant mémoriser

Compréhension des concepts fondamentaux et de la logique derrière chaque notion



Pratiquer le SQL

Exercices d'écriture de requêtes DDL, DML et DQL sur des bases de test



Refaire les schémas

Modélisation Entité-Association et passage au modèle relationnel



Résoudre des exercices

Exercices de normalisation, conception de schémas et d'optimisation



Travailler sur des cas réels

Projets appliqués et études de cas mettant en œuvre tous les concepts



Préparer l'examen

Répondre aux questions d'examen et simuler des situations pratiques



Conseil : La pratique régulière et la compréhension profonde sont clés pour réussir en bases de données

Bonnes Pratiques Professionnelles



Documentation Claire

- Schémas détaillés
- Requêtes commentées
- Contraintes explicites



Sécurité dès la Conception

- Privilèges minimaux
- Chiffrement des données
- Authentification robuste



Sauvegardes Régulières

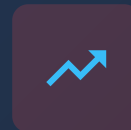
- Sauvegardes complètes
- Sauvegardes différentielles
- Sauvegardes incrémentielles



Tests Continus

Requêtes, transactions, intégrité

- Tests unitaires des fonctions SQL
- Tests d'intégrité référentielle
- Tests de performance



Optimisation Progressive

Indexation, requêtes, structure

- Création d'index sur colonnes fréquemment filtrées
- Optimisation des requêtes SELECT
- Revue et ajustement des schémas



Excellence : Ces pratiques garantissent des bases de données performantes, sécurisées et maintenables

Conclusion Générale du Cours

Les bases de données constituent le socle de l'informatique moderne



Gestion Traditionnelle

Les Igoudar marocains : premiers systèmes de stockage communautaire



SGBD Avancés

MySQL, Oracle : systèmes de gestion de bases de données modernes



Objectif : Stocker, Sécuriser et Exploiter l'Information Efficacement



Base Théorique

Concepts fondamentaux solides



Compétences Pratiques

Maîtrise de SQL et des SGBD



Méthodologie Professionnelle

Approche structurée et rigoureuse



Merci de votre attention !



Théorie

Concepts fondamentaux



Pratique

SQL et SGBD



Professionnel

Méthodologie



Conseil : Pratiquez régulièrement pour maîtriser les bases de données



Bonne chance dans vos projets de bases de données !

BASE DE DONNÉES



STORY MANGA COMICS



Un voyage explosif des données aux SGBD!





STORY MANGA COMICS

UN VOYAGE EXPLOSIF DES DONNÉES AUX SGBD!



SENSEI

Le Maître



YASSINE

Le Curieux



SARA

L'Appliquée



ROBO-DB

L'Assistant

SCÈNE 0 – LE DÉBUT DE L'AVENTURE!



SALLE DE CLASSE

SENSEI

Aujourd'hui, on commence un voyage : des données jusqu'aux bases de données modernes !

YASSINE

Ça va parler SQL ?

SENSEI

Oui... mais d'abord, on comprend les bases.

⚡ **WHOOOOSH!** ⚡

01

CHAPITRE 1 – DONNÉE VS INFORMATION!

🔗 LA RÉVÉLATION

ROBO-DB
Ahmed – 25 – INF102 – 14/09/2025

SARA
Euh... c'est quoi ça ?

SENSEI
Ça, c'est une DONNÉE. Un élément brut, sans contexte.

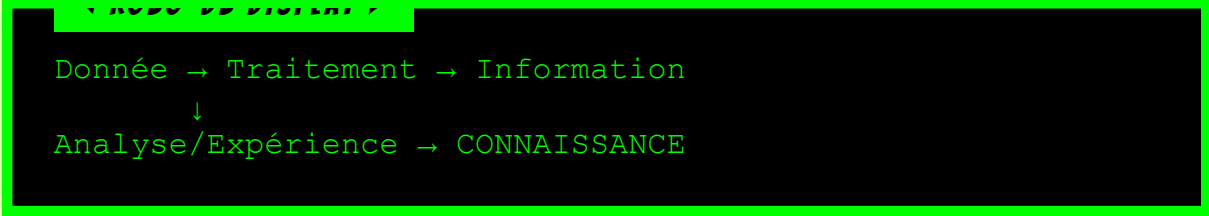
YASSINE
Donc seule, la donnée n'explique rien ?

💡 **EXACTEMENT!** 💡

SENSEI
Maintenant : Ahmed, âgé de 25 ans, est inscrit au module INF102 le 14/09/2025.

SARA
Là je comprends !

SENSEI
Ça, c'est une INFORMATION : données + organisation + contexte.



L'HISTOIRE D'IGOUDAR!



MAROC ANCESTRAL

SARA

Sensei, avant les ordinateurs, comment on g rait les donn es ?

SENSEI

Au Maroc, il existe un exemple fascinant : les IGOUDAR !

 **FLASH-BACK!** 

SENSEI

Chaque famille avait une cellule priv e, identifi e par un symbole ou un num ro.

YASSINE

Comme un casier s curis  !

SARA

Donc c' tait une banque communautaire !

```
+-----+
|   I G O U D A R   |
| [Fam A]   [Fam B] |
| [Fam C]   [Fam D] |
| [Fam E]   [Fam F] |
+-----+
```

```
Moderne      ↔ Igoudar
Base de donn es ↔ Igoudar
Enregistrement ↔ Cellule
S curit       ↔ Gardien
Admin         ↔ Amin
```

⚠️ PROBLÈMES DES FICHIERS! ⚠️

 ÈRE INFORMATIQUE

SENSEI

Au début, on stockait dans des FICHIERS...

ROBO-DB

etudiants.txt – notes.txt – enseignants.txt

SARA

Ça paraît simple...

💣 **DANGER!** 💣

ROBO-DB

Fichier 1 : Ahmed, 25 ans
Fichier 2 : Ahmed, 26 ans

SARA

Quelle version est correcte ?!

- ⚠️ PROBLÈMES :
- X Redondance
 - X Incohérence
 - X Dépendance forte
 - X Pas de sécurité
 - X Accès limité



LA SOLUTION : BASE DE DONNÉES!

★ DÉFINITION

SENSEI

Une BASE DE DONNÉES est un ensemble structuré de données, stockées durablement, avec accès efficace, sécurisé et partagé!

⚡ CARACTÉRISTIQUES ⚡

ROBO-DB

- ✓ Centralisée
- ✓ Cohérente
- ✓ Persistante
- ✓ Sécurisée
- ✓ Partagée

SARA

Ça sert où ?

SENSEI

PARTOUT! Banques, universités, hôpitaux, e-commerce, réseaux sociaux...

MODE DE VIE

Utilisateurs → Applications

↓

SGBD

↓

Base de Données

ACTEURS:

- DBA (Admin)
- Concepteur
- Développeur
- Utilisateur



CHAPITRE 2 – LE SGBD!

🔧 SYSTÈME DE GESTION

SENSEI

Une base seule ne suffit pas. Il faut un SGBD pour tout contrôler!

YASSINE

C'est le logiciel qui gère tout ?



EXACT!



SARA

Quelles fonctions ?

SENSEI

DDL : Définir (CREATE, ALTER, DROP)

DML : Manipuler (INSERT, UPDATE, DELETE)

DQL : Interroger (SELECT)

FONCTIONS SGBD:

- ✓ DDL (Definition)
- ✓ DML (Manipulation)
- ✓ DQL (Requêtes)
- ✓ Sécurité
- ✓ Concurrence
- ✓ Transactions
- ✓ Sauvegarde



ARCHITECTURES!



NIVEAUX

ROBO-DB

1 NIVEAU : tout sur une machine

SENSEI

Simple, mais limité!



ÉVOLUTION!



ROBO-DB

2 NIVEAUX : Client ↔ Serveur

3 NIVEAUX!

SENSEI

Architecture ANSI/SPARC : Indépendance totale des données!

ARCHITECTURE 3 NIVEAUX:

Externe (Vues)

↓

Conceptuel (Schéma)

↓

Interne (Stockage)

✓ Indépendance logique

✓ Indépendance physique

MODÉLISATION!

CONCEPTION

SENSEI

On MODÉLISE avant de coder : entités, attributs, associations!

DESIGN TIME!

SENSEI

Entité → TABLE

Association N,N → TABLE d'association

SARA

Pourquoi normaliser ?

ÉLIMINER REDONDANCE!

```
MODÈLE EA:
[ETUDIANT] -- S'INSCRIT -- [MODULE]
  (1,N)                (1,N)

RELATIONNEL:
ETUDIANT(#id, nom)
MODULE(#id, libelle)
INSCRIPTION(#etudiant, #module, note)

NORMALISATION: 1FN → 2FN → 3FN
```



SQL - LE LANGAGE!



CODING TIME

SENSEI

SQL : Le langage universel des bases de données!



ROBO-DB

CREATE / INSERT / SELECT / COMMIT



⚡ **CODE POWER!** ⚡

YASSINE

JOIN ? GROUP BY ?



SENSEI

Oui! On interroge et analyse avec SELECT!



~ ROBO-DB DISTRICT ~

SQL - STRUCTURED QUERY LANGUAGE

DDL: CREATE, ALTER, DROP

DML: INSERT, UPDATE, DELETE

DQL: SELECT, JOIN, GROUP BY

ACID:

- Atomicité
- Cohérence
- Isolation
- Durabilité

SGBD & SÉCURITÉ!

PROTECTION

SENSEI

MySQL : Open source, rapide, fiable!

SENSEI

Oracle : Puissance entreprise!

SÉCURITÉ CIA!

ROBO-DB

PL/SQL : Variables, IF, boucles, procédures, triggers!

SGBD POPULAIRES:

- MySQL (Open Source)
- Oracle (Enterprise)
- PostgreSQL
- SQL Server

SÉCURITÉ CIA:

- ✓ Confidentialité
- ✓ Intégrité
- ✓ Disponibilité



PROJET COMPLET - FIN!



VICTOIRE

SENSEI

Projet complet : Besoin → EA → Relationnel → SQL →
Tests → Sécurité!



SARA

Maintenant je comprends TOUT le processus!



YASSINE

De la donnée brute jusqu'à la base sécurisée!



MISSION ACCOMPLIE!



ROBO-DB

Vous êtes prêts pour créer vos propres bases
de données!



SENSEI

FÉLICITATIONS! Vous avez terminé ce voyage!



LA ROUTE DE VICTOIRE

CYCLE COMPLET:

1. Analyse besoins
2. Modèle EA
3. Modèle relationnel
4. Normalisation
5. Implémentation SQL
6. Tests
7. Sécurité
8. Maintenance



FIN DU COURS

TO BE CONTINUED...

THE END

MERCI D'AVOIR LU!